Spécifications Définitives : Système d'Annotations UML

Bibliothèque de Génération Java vers PlantUML

Documentation Architecturale

9 août 2026

Table des matières

1 Introduction **5**
- 1.1 Comment le code est lu . . . 5
- 1.2 Périmètre . . . 6
- 1.3 Principe directeur : la sobriété de l'API . . . 6

2 Règles générales et conventions **6**
- 2.1 Convention de nommage . . . 6
- 2.2 Comment un membre est écrit . . . 6
- 2.3 Un seul énuméré de direction . . . 8
- 2.4 Annotations répétables . . . 8
- 2.5 Cibles autorisées . . . 8
- 2.6 Résolution des conflits . . . 9

3 Le Déclencheur et le Seuil : `@UMLClass` **10**
- 3.1 Principe . . . 10
- 3.2 Le paramètre de Stratégie (Visibilité) . . . 10
- 3.3 Le filtre anti-bruit : rien n'est masqué sans demande . . . 10
- 3.4 Pourquoi c'est utile ? . . . 11
- 3.5 Paramètres . . . 11
- 3.6 Exemple . . . 11

4 La Dérogation d'Exclusion : `@UMLExclude` **12**
- 4.1 Principe . . . 12
- 4.2 Pourquoi c'est utile ? . . . 12
- 4.3 Paramètres . . . 12
- 4.4 L'exclusion se déclare là où le membre est déclaré . . . 12
- 4.5 Exemple . . . 12

5 La Dérogation d'Inclusion : `@UMLInclude` **12**
- 5.1 Principe . . . 12
- 5.2 Pourquoi c'est utile ? . . . 13
- 5.3 Paramètres . . . 13
- 5.4 Le renommage d'un membre . . . 13

5.5 La notation attribut d'un accesseur . . . 13
5.6 Exemple . . . 14

6 La Précision Métier : `@UMLAssociation` 14
6.1 Principe . . . 14
6.2 Pourquoi c'est utile ? . . . 14
6.3 Paramètres . . . 14
6.4 Exemple . . . 15
6.5 Les deux notations d'une association . . . 15
6.6 Deux attributs qui se font face ne sont qu'une association . . . 16

7 La Classe d'Association : `@UMLAssociationClass` 17
7.1 Principe . . . 17
7.2 Pourquoi c'est utile ? . . . 17
7.3 Paramètres . . . 17
7.4 Exemple . . . 18

8 La Multiplicité d'une propriété : `@UMLMultiplicity` 18
8.1 Principe . . . 18
8.2 Pourquoi c'est utile ? . . . 18
8.3 Ce qu'elle ne déduit pas, et ce que le projet peut traduire . . . 19
8.4 Paramètres . . . 19
8.5 Combinaison avec les autres annotations . . . 19

9 Les Éléments Dérivés : `@UMLDerived` 19
9.1 Principe . . . 19
9.2 Pourquoi c'est utile ? . . . 20
9.3 Paramètres . . . 20
9.4 Exemple . . . 20

10 Les Contraintes Métier : `@UMLConstraint` 20
10.1 Principe . . . 20
10.2 Pourquoi c'est utile ? . . . 20
10.3 Paramètres . . . 21
10.4 Où la contrainte est écrite . . . 21
10.5 Exemple . . . 21

11 Les Liens Transversaux : `@UMLDependency` 21
11.1 Principe . . . 21
11.2 Pourquoi c'est utile ? . . . 21
11.3 Paramètres . . . 21
11.4 Exemple . . . 22

12 La Documentation Visuelle : `@UMLNote` 22
12.1 Principe . . . 22
12.2 Pourquoi c'est utile ? . . . 22
12.3 Paramètres . . . 22
12.4 Ce à quoi une note peut s'accrocher . . . 22
12.5 Exemple . . . 23

13 Le Regroupement Logique : `@UMLPackage` 23
13.1 Principe . . . 23
13.2 Pourquoi c'est utile ? . . . 23
13.3 Paramètres . . . 23
13.4 Exemple . . . 24

14 Le Correcteur de Mise en Page : `@UMLLayout` 24
14.1 Principe . . . 24
14.2 Pourquoi c'est utile ? . . . 24
14.3 Paramètres . . . 24
14.4 Exemple . . . 24

15 Les règles du projet : le fichier `umlg.rules` 25
15.1 Principe . . . 25
15.2 Pourquoi un fichier, et non une annotation ? . . . 25
15.3 Forme du fichier . . . 25
15.4 Le vocabulaire complet . . . 27
15.5 Un fichier complet . . . 28
15.6 `strategy` : la visibilité de tout le projet . . . 28
15.7 `hide` : ce que le projet tient pour du bruit . . . 28
15.8 `type` et `hide return` : les types affichés . . . 29
15.9 `title`, `direction`, `show empty members` : le dessin d'ensemble . . . 29
15.10 `group package` : ranger un paquetage . . . 29
15.11 Ce que les règles ne touchent pas . . . 30
15.12 Ce qui est refusé . . . 30
15.13 Une extension prévue . . . 32

16 L'apparence du diagramme : `UmlStyle` 32
16.1 Principe . . . 32
16.2 Pourquoi ce n'est pas une annotation . . . 32
16.3 Les deux valeurs . . . 32
16.4 Ce qui échappe au choix . . . 33

17 Les vues : plusieurs diagrammes pour un projet 33
17.1 Principe . . . 33
17.2 Déclarer une vue . . . 33
17.3 L'ordre d'écriture est l'ordre de lecture . . . 34
17.4 Les vues calculées . . . 35
17.5 Les liens qui sortent d'une vue . . . 35
17.6 Passer d'une vue à l'autre . . . 35
17.7 Quand le dessin devient illisible . . . 35
17.8 Ce que la génération dit d'elle-même . . . 36
17.9 Ce que la génération signale . . . 37
17.10 Ce que devient la génération précédente . . . 39
17.11 Lancer la génération . . . 39
17.12 Demander un dossier . . . 40

18 Les boîtes que le code ne porte pas **41**
18.1 Principe 41
18.2 Ce que cela coûte, et comment on le paie 41
18.3 Ce qui est refusé 41

19 La vérification à la compilation **41**
19.1 Principe 41
19.2 Pourquoi c'est utile ? 41
19.3 Deux gravités, et une règle pour les départager 42
19.4 Ce qui arrête la compilation 43
19.5 Ce qui produit un avertissement 45
19.6 Faire taire les avertissements 46
19.7 Donner le fichier de règles au compilateur 46

20 Choix de conception : ce qui est hors périmètre **47**
20.1 Fonctionnalités écartées 47
20.2 Ce qui relève de l'analyse automatique, non des annotations 47
20.3 Ce qui ne peut pas être déduit, et pourquoi 48

1 Introduction

Ce document référence le système d'annotations Java conçu pour la génération automatique de diagrammes PlantUML. L'outil a été pensé pour allier la simplicité de la déduction automatique du code Java avec la flexibilité requise pour modéliser des concepts métier complexes et maîtriser le rendu visuel. Il gère nativement les concepts standards (héritage via `extends`, implémentation via `implements`, classes abstraites et énumérations). Les annotations ci-dessous viennent enrichir et paramétrer cette analyse.

1.1 Comment le code est lu

La bibliothèque travaille **par réflexion sur les classes compilées** : on lui désigne un répertoire de `.class` (typiquement `target/classes`), et non un répertoire de sources. Ce choix a une conséquence qu'il faut énoncer d'emblée, car elle explique la forme de plusieurs annotations : **ce que la compilation efface est définitivement perdu pour le générateur**. Ainsi les valeurs d'initialisation des attributs n'existent plus, et le corps des méthodes non plus — une dépendance qui ne se lit que dans un appel devra être déclarée.

Ce que la compilation n'efface pas. Une idée reçue mérite d'être écartée ici, car elle a longtemps guidé la conception de cette bibliothèque : on lit souvent que l'effacement de type (*type erasure*) fait disparaître les paramètres de type, et qu'un `List<Commande>` deviendrait un `List`. C'est vrai de ce qui vit à l'exécution — deux listes, l'une de commandes et l'autre de clients, sont le même objet en mémoire, et aucune ne sait ce qu'elle contient. Ce n'est pas vrai des **signatures déclarées** : le compilateur écrit dans le `.class` un attribut `Signature` qui conserve le type tel qu'il a été écrit, précisément pour que d'autres compilations puissent le relire. La réflexion l'expose (`getGenericType`, `getGenericReturnType`), et la bibliothèque s'en sert : un attribut déclaré `Map<String, Short>` s'écrit `Map<String, Short>` sur le diagramme, et un `List<Commande>` dit à lui seul quelle classe l'association vise.

Loin d'être une limite subie, c'est le fondement du principe retenu : **rien n'est deviné, tout est déclaré**. La classe située à l'autre bout d'un lien et la multiplicité de ce lien sont des paramètres obligatoires ou explicites de `@UMLAssociation`, et non des déductions. Une déduction automatique serait de toute façon incapable de trancher entre `0..*` et `1..*`, qui relèvent du métier et non du code ; la faire disparaître ne coûte donc rien et supprime une source d'écarts silencieux entre le code et le dessin.

Une seconde lecture, indépendante, a lieu **au moment de la compilation** : la bibliothèque embarque un processeur d'annotations qui vérifie ce que le développeur écrit, là où il l'écrit (voir la section 19).

Une exception mérite d'être signalée, car elle ne dépend pas de la bibliothèque mais du projet analysé : les **noms des paramètres de méthode** ne sont écrits dans les `.class` que si celui-ci est compilé avec l'option `-parameters`. Le générateur les affiche lorsqu'ils existent, et se limite aux types sinon — en le **signalant** : quand aucun paramètre d'aucune méthode ne porte de nom, la génération avertit que l'option manque, plutôt que de laisser l'utilisateur découvrir des signatures amputées sans indice sur leur cause (section 2.2).

1.2 Périmètre

La bibliothèque cible exclusivement le **diagramme de classes**. Les autres types de diagrammes supportés par PlantUML (séquence, composants, états, cas d'utilisation) sont hors périmètre et ne font l'objet d'aucune annotation.

1.3 Principe directeur : la sobriété de l'API

La valeur de la bibliothèque ne se mesure pas au nombre de fonctionnalités PlantUML rendues accessibles, mais au **nombre d'annotations qu'un développeur doit écrire pour obtenir un diagramme correct**. Dans le cas nominal, `@UMLClass` doit suffire.

Ce principe a une conséquence directe : toute annotation supplémentaire alourdit le code source, augmente le coût d'apprentissage et multiplie les interactions à spécifier. Un jeu d'annotations trop riche déplacerait le problème au lieu de le résoudre — on écrirait alors davantage de spécification UML que de Java, et le développeur retournerait rédiger son fichier `.puml` à la main. Les directives de rendu purement esthétiques sont donc volontairement écartées (voir la section 20).

Il en découle un second partage, entre ce qui se déclare sur un élément et ce qui vaut pour tout le projet. Une annotation dit une chose sur *cet* attribut ou *cette* classe ; ce qui s'applique partout — par exemple qu'un `ImmutableList` s'affiche `List` — se déclare une seule fois dans le fichier `umlg.rules` (section 15), et non sur chaque membre concerné.

2 Règles générales et conventions

Cette section fixe les règles transverses valables pour l'ensemble des annotations. Elle lève les ambiguïtés de combinaison qui, sans arbitrage explicite, rendraient le rendu imprévisible.

2.1 Convention de nommage

Tout paramètre désignant une classe visée par un lien s'appelle `target` (et `source` pour l'extrémité de départ). Cette convention est uniforme sur `@UMLAssociation`, `@UMLAssociationClass`, `@UMLDependency` et `@UMLLayout`.

2.2 Comment un membre est écrit

Le générateur emploie la notation d'UML, `nom : Type`, et non l'ordre de déclaration de Java. Le nom vient d'abord, le type après les deux-points : c'est l'ordre dans lequel un diagramme se lit, où l'on cherche une propriété par son nom et non par son type. La règle vaut aux trois endroits où un type paraît — un attribut, un paramètre, le retour d'une méthode. Les deux-points sont **entourés d'espaces**, comme les écrit la norme UML : `solde : double`, et non `solde: double`, qui est la ponctuation de Java et de PlantUML.

```
class "Vol" {
  - departureTime : Date
  - escales : Aeroport [0..*]
  --
  + reporter(nouvelleHeure : String, minutes : int) : void
  + {static} /compter(code : String) : int
}
```

L'ordre des morceaux est fixe. Pour un attribut : la visibilité, `{static}` s'il y a lieu, le nom (précédé de sa barre oblique s'il est dérivé), le type, la multiplicité entre crochets, puis les contraintes entre accolades. Pour une méthode : la visibilité, `{static}`, le nom, les paramètres entre parenthèses, le type de retour, puis sa multiplicité.

Le type de retour. Il est écrit par défaut, `void` compris. UML le tient pourtant pour facultatif, et l'omettre veut dire « ne rend rien » — c'est la forme qu'emploient la plupart des diagrammes tenus à la main. Le choix se déclare une fois pour tout le projet, par la règle `hide return void` (section 15), et non annotation par annotation : il tient à la façon dont on veut lire le diagramme, jamais à telle méthode en particulier.

Les noms des paramètres. Ils ne sont affichés que si le projet analysé a été compilé avec l'option `-parameters` ; sans elle, la réflexion ne rend que `arg0`, `arg1`…, que le générateur **n'écrit jamais** — le lecteur croirait lire le code de l'auteur. La signature se réduit alors à `reporter(String, int): void`. Pour Maven, l'option se pose sur le `maven-compiler-plugin` :

```
<configuration>
    <parameters>true</parameters>
</configuration>
```

Cette dégradation était longtemps **silencieuse** : rien ne distinguait, sur le dessin, un projet compilé sans l'option d'un projet aux signatures naturellement sobres. La génération la **signale** désormais — quand au moins une méthode dessinée a des paramètres et qu'aucun n'a de nom, un avertissement nomme l'option manquante et la ligne de pom à ajouter. La condition est stricte pour ne jamais accuser à tort : un projet compilé avec l'option a *tous* les noms, en voir un seul suffit à se taire ; et un projet qui les a masqués par la règle `hide parameter names` (ci-dessous) les a tus volontairement, et n'est pas signalé.

La forme courte peut aussi être un **choix** : la plupart des diagrammes tenus à la main écrivent `from(KeyboardMap)` et non `from(keyboardMap : KeyboardMap)`, et les signatures complètes font des boîtes larges — mesuré sur un projet réel, une vue passait du simple au double. La règle `hide parameter names` du fichier `umlg.rules` (section 15) demande les types seuls, même quand la compilation a gardé les noms.

L'ordre des membres. Par défaut, les attributs et les méthodes d'une boîte sont rangés par **ordre alphabétique**. Ce n'est pas un choix esthétique mais une exigence de reproductibilité : la réflexion rend les membres « dans un ordre non spécifié », qui varie réellement d'une exécution à l'autre, et deux générations du même code doivent produire le même fichier. L'ordre de déclaration du source, plus naturel à lire, ne lui est pourtant plus interdit : la règle `order declaration` (section 15) le rétablit en relisant les tables de membres du fichier `.class`, que les compilateurs écrivent dans l'ordre du source — un ordre que le fichier, lui, ne change pas entre deux lectures, ce qui préserve la reproductibilité. Les littéraux d'une énumération font exception dans les deux cas : leur ordre est celui des ordinaux, qui a un sens que l'alphabet détruirait.

Le type devant une multiplicité. Dès qu'une multiplicité est écrite, le type qui la précède est celui de l'**élément**, jamais celui du conteneur. Un attribut déclaré `Set<String>` et marqué `[0..*]` s'écrit `alias: String [0..*]`, et non `alias: Set [0..*]` — lequel

se lirait « zéro ou plusieurs ensembles », ce qui est faux : il y a un ensemble, contenant zéro ou plusieurs chaînes. `Optional<Integer>` suit la même règle. Une `Map`, en revanche, a deux paramètres et rien ne dirait lequel est la propriété : elle s'écrit en entier.

La multiplicité. Elle n'est écrite que là où elle est déclarée : par `targetCardinality` sur une association dessinée dans la boîte (`notation = ATTRIBUTE`, section 6.5), ou par `@UMLMultiplicity` sur n'importe quel attribut ou méthode (section 8). Une multiplicité absente vaut *exactement un*, et ne s'écrit donc pas.

2.3 Un seul énuméré de direction

L'énuméré `UMLDirection` (`NONE`, `UP`, `DOWN`, `LEFT`, `RIGHT`) est utilisé partout où une orientation est demandée. `NONE` est le défaut : aucune contrainte n'est posée et PlantUML place l'élément lui-même, ce qui est le bon choix dans la quasi-totalité des cas. Pour une note, `UP` et `DOWN` correspondent respectivement au placement *au-dessus* et *en dessous* de l'élément annoté.

2.4 Annotations répétables

Une classe dépend fréquemment de plusieurs autres, porte plusieurs notes ou plusieurs contraintes. Les annotations suivantes sont donc déclarées `@Repeatable` et peuvent être apposées plusieurs fois sur le même élément :

- `@UMLDependency`
- `@UMLNote`
- `@UMLConstraint`
- `@UMLLayout`

Les autres annotations (`@UMLClass`, `@UMLAssociation`, `@UMLAssociationClass`, `@UMLMultiplicity`, `@UMLPackage`, `@UMLDerived`, `@UMLInclude`, `@UMLExclude`) sont uniques sur leur cible.

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
@UMLDependency(target = Journal.class, label = "trace")
@UMLClass
public class Panier { }
```

2.5 Cibles autorisées

Cible	Annotations acceptées
Type (classe, interface, énumération)	`@UMLClass`, `@UMLPackage`, `@UMLAssociationClass`, `@UMLDependency`, `@UMLLayout`, `@UMLNote`, `@UMLConstraint`
Attribut	`@UMLInclude`, `@UMLExclude`, `@UMLAssociation`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLConstraint`, `@UMLNote`
Méthode	`@UMLInclude`, `@UMLExclude`, `@UMLMultiplicity`, `@UMLDerived`, `@UMLNote`

Une annotation posée hors de ces cibles est rejetée à la compilation.

2.6 Résolution des conflits

— `@UMLInclude` **et** `@UMLExclude` **sur le même élément : erreur de compilation**. La bibliothèque saurait départager — la dissimulation l'emporterait — mais appliquer une règle d'arbitrage en silence reviendrait à choisir à la place de l'auteur, dans un code qui se contredit. Le message invite à ne garder que celle des deux qui dit ce que l'on veut.

— `@UMLInclude(name = ...)` **et** `targetRole` **sur le même attribut, sous** `notation = ATTRIBUTE` **: erreur de compilation**. En notation attribut, le rôle cible *est* le nom de la ligne ; deux noms pour la même ligne se contrediraient, et rien ne dirait lequel fait foi.

— **Classe visée par un lien mais dépourvue de** `@UMLClass` **: erreur de compilation**, positionnée sur le paramètre `target` fautif. Le message nomme l'endroit (la classe et l'attribut porteurs), la classe visée, et les deux issues possibles : annoter la cible, ou viser une classe qui l'est déjà. Une cible est en effet toujours écrite exprès : son absence du diagramme est un oubli, jamais une intention.

Une seule exception : les liens d'héritage, qui ne sont pas déclarés mais lus dans la signature de la classe. Une classe mère hors du diagramme y est le cas courant — tout hérite d'`Object`, beaucoup de classes implémentent `Comparable` — et le lien est alors **ignoré silencieusement**. Seul le périmètre déclaré par `@UMLClass` fait foi ; aucune classe n'est créée implicitement, ce qui garantit que le diagramme ne déborde jamais du périmètre choisi.

— `@UMLConstraint` **sur un attribut porteur de** `@UMLAssociation` : la contrainte s'applique à l'**association** et non à l'attribut ordinaire, ce qui correspond à l'usage majoritaire, du type `{subset}` ou `{ordered}`. Elle est écrite là où l'association est dessinée : à l'extrémité du trait sous `notation = ARROW`, à la suite de la ligne dans la boîte sous `ATTRIBUTE`. En l'absence de `@UMLAssociation`, elle s'applique à l'attribut.

— `@UMLDerived` **sur un attribut porteur de** `@UMLAssociation` : c'est l'association qui est marquée comme dérivée, la barre oblique se plaçant devant le nom que porte la propriété sur le dessin. Sous `notation = ARROW`, ce nom est le **rôle cible**, l'attribut ayant quitté la boîte pour devenir un trait ; si `targetRole` n'est pas renseigné, la propriété n'est nommée nulle part, il n'y a rien à marquer, et le compilateur avertit. Sous `ATTRIBUTE`, la barre retrouve sa place ordinaire devant le nom de la ligne, qui existe toujours : il n'y a alors rien à signaler.

— `@UMLExclude` **sur un attribut porteur de** `@UMLAssociation` : ce qui disparaît est ce que l'association demandait — le trait entre les deux boîtes sous `ARROW`, la ligne dans la boîte sous `ATTRIBUTE`. Dans les deux cas le compilateur avertit, l'effet dépassant ce qu'un lecteur pressé attendrait.

— **Stratégie de visibilité et** `@UMLAssociation` : une association est **toujours** dessinée, quelle que soit la `strategy` de la classe et la visibilité de l'attribut porteur. Une demande explicite l'emporte sur un défaut ; la règle vaut pour les deux notations, si bien que passer de l'une à l'autre ne fait jamais apparaître ni disparaître un lien.

— **Deux classes déclarant le même** `@UMLPackage(name)` **avec des couleurs différentes** : la première couleur rencontrée dans l'ordre d'analyse est retenue et un avertissement est émis.

3 Le Déclencheur et le Seuil : `@UMLClass`

3.1 Principe

Placée sur une classe, interface ou énumération, cette annotation indique que l'entité doit figurer sur le diagramme. Elle permet également de définir un seuil de visibilité par défaut pour ses attributs et méthodes, agissant comme un filtre d'inclusion.

3.2 Le paramètre de Stratégie (Visibilité)

Le paramètre `strategy` définit exactement quels niveaux d'encapsulation doivent être affichés automatiquement :

- `PROJECT` **(par défaut)** : La classe s'en remet à la ligne `strategy` du fichier `umlg.rules` (section 15), ou à `NONE` si le projet n'en déclare aucune. C'est ce qui permet d'écrire la stratégie une fois pour tout le projet plutôt que sur chacune des classes.
- `NONE` : La classe apparaît sur le diagramme comme une coquille vide (aucun attribut ni méthode n'est comptabilisé), sauf les éléments explicitement marqués `@UMLInclude`. Écrite sur une classe, elle l'emporte sur le réglage du projet : c'est ainsi qu'on ramène une boîte à son seul nom dans un diagramme qui montre le reste.
- `PUBLIC` : Seuls les éléments `public` sont générés.
- `PROTECTED` : Les éléments `public` et `protected` sont générés.
- `PACKAGE` : Les éléments `public` et ceux sans modificateur (package-private) sont générés. Les éléments `protected` sont exclus par défaut.
- `PRIVATE` : Tous les éléments de la classe sont générés, sans aucune restriction.

Les composants d'un record échappent à la stratégie. `record Mnemonic(String keysymMnemonic)` ne dit rien d'autre que ce composant : il **est** la définition du record, et une boîte qui le tairait serait vide de ce qu'elle est. Les composants se dessinent donc quelle que soit la stratégie — `NONE` comprise, qui est faite pour choisir parmi des membres ordinaires, pas pour taire une définition. `@UMLExclude`, veto individuel, garde le dernier mot ; un champ *ajouté* au record (forcément statique) reste un membre ordinaire, soumis à la stratégie. Mesuré sur un projet réel, cet assouplissement a retiré onze lignes d'annotations de trois records triviaux, dont la seule fonction était de faire paraître le composant et de taire le reste.

3.3 Le filtre anti-bruit : rien n'est masqué sans demande

Une classe réelle a plus de membres qu'un diagramme lisible n'en supporte, et certains n'apprennent rien — un `getSolde()` ne dit pas davantage que l'attribut `solde`. La bibliothèque sait les reconnaître, mais **elle n'en masque aucun de sa propre initiative** : le tri se déclare dans le fichier `umlg.rules` (section 15), et rien ne s'applique tant que rien n'y est écrit.

Ce n'est pas une commodité mais un choix de fond. Le jugement « ceci n'apprend rien » est bon dans beaucoup de projets et mauvais dans d'autres : sur un code immuable écrit à la manière de Guava, il n'y a pas un accesseur à écarter, et ce qui encombre les boîtes est ailleurs — des surcharges, des fabriques. Un filtre qui décide seul retire alors du diagramme des membres que personne n'avait demandé à cacher, sans que rien ne le

signale. Chaque projet dit donc ce qu'il tient pour du bruit ; la bibliothèque ne fait que l'appliquer.

Une exception, et une seule : ce que le compilateur a fabriqué et que le développeur n'a jamais écrit — les attributs synthétiques, la référence à la classe englobante d'une classe interne. Cela n'a jamais été un jugement de valeur, et reste écarté sans qu'on ait à le demander.

Les constructeurs, eux, ne sont pas filtrés : ils ne sont jamais lus. Aucune annotation ne peut donc en faire apparaître un.

3.4 Pourquoi c'est utile ?

Sans cette annotation, le générateur tenterait de dessiner toutes les classes du projet, y compris les classes utilitaires ou les dépendances externes. Elle permet au développeur de choisir exactement le périmètre de son architecture.

3.5 Paramètres

- `name` (String) : Surcharge le nom affiché (utile pour raccourcir les noms à rallonge).
- `color` (String) : Applique une couleur de fond — un code hexadécimal (`"#DDEEFF"`, `"#ABC"`, `"#FF000080"` avec transparence) ou un nom de couleur PlantUML (`"LightBlue"`). Le catalogue des noms, longtemps enfermé dans un paquetage non exporté, s'obtient par `UMLGenerator.couleursConnues()` : l'utilisateur dont la couleur est refusée peut demander lesquelles existent, et le message de refus l'y renvoie. La question est posée au catalogue réel de PlantUML, et à lui seul — une grammaire écrite en doublon côté extraction paraissait plus sobre, mais elle divergeait : elle refusait `"#FF000080"`, que le moteur accepte, si bien que le processeur d'annotations laissait compiler du code qui ne pouvait pas se dessiner. Une seule grammaire, celle du moteur, et l'écart ne peut plus exister.
- `stereotype` (String) : Ajoute une étiquette métier (ex : `"Service"`). Les guillemets français « » sont posés par le générateur : les écrire soi-même les afficherait en double, ce dont le compilateur avertit.
- `strategy` (Enum) : `PROJECT` (défaut, renvoie au fichier de règles), `NONE`, `PUBLIC`, `PROTECTED`, `PACKAGE`, `PRIVATE`.

3.6 Exemple

```
// Affiche automatiquement les elements autorises, mais filtre le bruit
@UMLClass(stereotype = "Entity", strategy = UMLVisibility.PACKAGE)
public class Utilisateur {
    public String pseudo;        // Sera affiche
    String email;                // Sera affiche (package-private)
    protected int age;           // Sera ignore (hors strategie)
    private String motDePasse;   // Sera ignore (hors strategie)

    public Utilisateur() {}      // Sera ignore (Filtre anti-bruit : constructeur)
    public String getPseudo() {}// Sera ignore (Filtre anti-bruit : getter)

    public void valider() {}     // Sera affiche (methode metier publique)
```

```
}
```

4 La Dérogation d'Exclusion : `@UMLExclude`

4.1 Principe

Cette annotation est une exception négative. Elle force la dissimulation d'un attribut ou d'une méthode, même si la `strategy` de la classe l'autorisait à apparaître.

4.2 Pourquoi c'est utile ?

Dans le cas où une classe possède une `strategy = PRIVATE` (tout afficher), certaines variables techniques (loggers, constantes internes) n'ont aucune valeur architecturale ou doivent être cachées. Les exclure permet d'alléger le diagramme final.

4.3 Paramètres

Aucun.

4.4 L'exclusion se déclare là où le membre est déclaré

Posée sur la déclaration d'une méthode — dans une interface, ou dans une classe parente —, l'annotation vaut pour **chacune de ses redéfinitions**. Le besoin vient d'un projet réel : une méthode `asString()` déclarée par une interface et réalisée par trois records devait être exclue *trois fois*, une par override, sans que la répétition dise rien de plus que la première annotation. Le membre s'exclut désormais une fois, là où il est déclaré ; une méthode statique ou privée, qui ne redéfinit rien, ne regarde qu'elle-même.

4.5 Exemple

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class ConnexionBase {
    private String url; // Affiche car strategie globale sur PRIVATE

    @UMLExclude
    private Logger log; // Ignore explicitement pour alleger le graphe
}
```

5 La Dérogation d'Inclusion : `@UMLInclude`

5.1 Principe

Cette annotation agit comme une exception positive à la règle de visibilité dictée par `@UMLClass`. Elle force l'apparition d'un attribut ou d'une méthode sur le diagramme, même si son niveau d'encapsulation aurait dû le masquer. Elle peut de surcroît **renommer** le membre qu'elle affiche, par son paramètre `name`.

5.2 Pourquoi c'est utile ?

Si une classe n'a aucune `strategy` définie (classe vide par défaut), ou une `strategy = PUBLIC`, le développeur peut cibler très précisément les éléments internes (privés) cruciaux pour la compréhension de l'architecture sans avoir à exposer le reste.

5.3 Paramètres

— `name` (String) : Le nom sous lequel le membre paraît dans la boîte, à la place du sien. La chaîne vide — le défaut — veut dire « le nom du code ».
— `asAttribute` (boolean, `false` par défaut) : Écrit la méthode en **notation attribut** — `xKeycodeByXKeyName : Map<String, Short>`, sans parenthèses, dans le compartiment des attributs.

5.4 Le renommage d'un membre

Le besoin vient des diagrammes de référence, qui nomment souvent les choses plus longuement que le code : un attribut `codeByName` qu'un README appelle `xKeycodeByXKeyName`. Sans renommage, le diagramme engendré ne peut jamais rejoindre le diagramme voulu — il faudrait renommer le code lui-même, ce qui n'est pas à un outil de dessin d'en décider.

Le renommage est porté par la dérogation d'affichage, et non par une règle du projet, pour la même raison que `@UMLClass(name = ...)` : c'est une décision sur *un* élément, écrite là où l'élément est écrit. Les notes (`@UMLNote`) suivent le nom affiché, comme elles suivent déjà le rôle cible d'une association en notation attribut.

Trois écarts sont vérifiés à la compilation : un nom fait d'espaces est une **erreur** (la ligne serait au nom invisible) ; `name` et `targetRole` sur la même ligne sont une **erreur** (deux noms se contrediraient, section 2) ; un nom identique à celui du code est un **avertissement** (le renommage n'a pas eu lieu, et l'auteur le chercherait sur l'image). Enfin, un `@UMLInclude` qui renomme n'est jamais signalé « sans effet » : renommer *est* un effet, même quand la stratégie affichait déjà le membre.

5.5 La notation attribut d'un accesseur

Un accesseur sans paramètre expose une **propriété** de l'objet, et les diagrammes qui montrent une structure l'écrivent comme telle : `xKeycodeByXKeyName : Map<String, Short>`, sans parenthèses, parmi les attributs. Le dessiner comme une méthode dit « comment on l'obtient » là où le diagramme veut dire « ce que c'est ». `asAttribute = true` opère cette conversion : la ligne quitte le compartiment des méthodes pour celui des attributs, garde son soulignement si la méthode est statique, sa barre oblique si elle est dérivée, et sa multiplicité si elle en porte une.

La décision est **individuelle**, et non une règle du projet, parce que le même diagramme mélange les deux formes : le diagramme de référence de Keyboardd écrit ses trois accesseurs de tables en notation attribut mais garde `latest(): Xkeys` avec ses parenthèses — une fabrique n'est pas une propriété, et seul l'auteur sait laquelle est laquelle. Une règle globale « tout accesseur sans paramètre devient attribut » aurait précisément converti les fabriques avec le reste, et manqué le diagramme visé.

La notation exige une méthode **sans paramètre** qui **rend quelque chose** : les parenthèses retirées, il ne reste que le nom et le type, et il faut que les deux existent. Les

trois écarts — des paramètres, un retour `void`, l'annotation posée sur un attribut, qui en est déjà un — sont des **erreurs** de compilation (section 19).

5.6 Exemple

```java
@UMLClass(strategy = UMLVisibility.PUBLIC)
public class MoteurDeRecherche {

    public void chercher() {} // Affiche car public

    private int compteur; // Ignore car private

    @UMLInclude
    private void algorithmeSecret() {} // Force l'affichage grace a l'
    annotation

    @UMLInclude(name = "xKeycodeByXKeyName")
    public Map<String, Short> codeByName() { ... }
    // Affiche : xKeycodeByXKeyName() : Map<String, Short>
}
```

6 La Précision Métier : `@UMLAssociation`

6.1 Principe

Surcharge la déduction automatique des relations portées par les attributs, en précisant la sémantique UML stricte, les cardinalités, les rôles et la classe ciblée.

6.2 Pourquoi c'est utile ?

Le code Java `List<Commande> commandes` indique qu'il y a plusieurs commandes. Mais seul le métier sait si c'est `0..*` ou `1..*`, et si c'est une simple association ou une composition forte. De plus, préciser explicitement la classe cible permet de pallier les limites de l'effacement de type (Type Erasure) sur les collections génériques.

6.3 Paramètres

— `target` (Class) : Pointe vers la classe cible. **Facultatif** : un attribut déclaré `List<Commande>` nomme déjà sa cible, et la bibliothèque la lit (section 1.1). Il reste nécessaire là où le type ne dit rien — une collection écrite sans paramètre, une `Map` dont rien ne dirait laquelle des deux moitiés est visée, un lien vers une classe qui n'apparaît pas dans le type. Ce n'est pas une entorse au principe « rien n'est deviné » : lire `List<Commande>` n'est pas deviner, c'est lire ce que l'auteur a écrit.

— `notation` (Enum) : `UMLNotation` — `ARROW` (défaut) ou `ATTRIBUTE`. Décide de la **forme** du dessin : un trait entre deux boîtes, ou une ligne dans celle de départ (section 6.5).

— `type` (Enum) : `ASSOCIATION`, `COMPOSITION`, `AGGREGATION`.

— `sourceCardinality` (String) : Multiplicité côté source (ex : `"1"`, `"0..1"`).

— `targetCardinality` (String) : Multiplicité côté cible (ex : `"0..*"`, `"*"`).

— `sourceRole` et `targetRole` (String) : Noms des rôles affichés aux extrémités du lien.
— `isOrdered` (boolean) : Si vrai, ajoute la contrainte de collection `{ordered}`.
— `qualifier` (String) : Génère une association qualifiée (le petit carré contenant une clé d'identification).
— `label` (String) : Texte sur la flèche (ex : `"possede"`).
— `direction` (Enum) : `UMLDirection` — `UP, DOWN, LEFT, RIGHT`.

6.4 Exemple

```java
public class Utilisateur {
    @UMLAssociation(
        target = Commande.class,
        type = UMLRelationType.COMPOSITION,
        sourceCardinality = "1",
        targetCardinality = "0..*",
        targetRole = "achats",
        isOrdered = true,
        label = "passe"
    )
    private List<Commande> commandes;
}
```

6.5 Les deux notations d'une association

UML offre deux façons de dessiner une même propriété, et n'en autorise **qu'une à la fois** : ou bien un trait entre deux boîtes, ou bien une ligne dans la boîte de départ. Les deux disent exactement la même chose ; le choix relève de la lisibilité, non de la sémantique. Le paramètre `notation` le rend explicite, et la bibliothèque se charge de tenir la règle du non-doublon — sous `ARROW` l'attribut est retiré de la boîte au profit du trait, sous `ATTRIBUTE` il y reste et aucun trait n'est tracé.

Valeur	Ce qui est dessiné	Quand la choisir
`ARROW` (défaut)	Un trait décoré à ses extrémités : `Client "1" *-- "achats 0..*" Commande`	Les associations qui structurent le domaine, celles que le lecteur doit voir en premier.
`ATTRIBUTE`	Une ligne du compartiment des attributs : `- commandes: Commande [0..*]`	Quand une classe en cite dix autres, ou quand la cible n'a rien d'intéressant à montrer d'elle-même : la toile d'araignée disparaît.

```java
@UMLClass
public class Client {
    // Un trait vers Commande (comportement par defaut).
    @UMLAssociation(target = Commande.class, targetCardinality = "1")
    private Commande courante;

    // Une ligne dans la boite : "- archivees: Commande [0..*]"
    @UMLAssociation(target = Commande.class, targetCardinality = "0..*",
```

```
                        notation = UMLNotation.ATTRIBUTE)
    private List<Commande> archivees;
}
```

Ce que la notation `ATTRIBUTE` retient. Une ligne de boîte ne porte qu'un nom, un type et une multiplicité. Trois paramètres y trouvent donc leur place, et trois seulement :

- `target` donne le **type** affiché — la classe visée, et non `List`, qui ne dirait rien du contenu ;
- `targetCardinality` donne la **multiplicité**, écrite entre crochets ;
- `targetRole` donne le **nom** de la propriété, s'il est renseigné : en notation attribut, le rôle *est* le nom que l'on lit. À défaut, c'est le nom de l'attribut Java.

S'y ajoutent, comme pour n'importe quelle ligne, ce que portent les autres annotations : la visibilité et `{static}` lus sur l'attribut, la barre oblique de `@UMLDerived`, `{ordered}` puis les `@UMLConstraint`.

```
@UMLConstraint(rule = "non vide")
@UMLDerived
@UMLAssociation(target = Commande.class, targetCardinality = "1..*",
                isOrdered = true, notation = UMLNotation.ATTRIBUTE)
protected List<Commande> achats;
// Ligne produite : # /achats: Commande [1..*] {ordered} {non vide}
```

Ce qu'elle laisse de côté. Les paramètres qui décorent un *trait* n'ont alors nulle part où s'écrire : `type` (le losange d'une composition), `sourceCardinality`, `sourceRole`, `qualifier`, `label` et `direction`. Ce n'est pas une faute — le diagramme reste valide — mais l'auteur a écrit ces valeurs pour obtenir un effet qu'il n'aura pas : le compilateur les lui nomme (section 19).

Ce qu'elle ne relâche pas. La classe visée doit porter `@UMLClass` dans les deux notations, et la grammaire des multiplicités reste la même. La notation choisit une forme de dessin ; elle ne change aucune exigence sur ce qui est dessiné.

6.6 Deux attributs qui se font face ne sont qu'une association

Un modèle métier est plein d'attributs réciproques : `Eleve` cite `Cours`, et `Cours` cite `Eleve`. Chaque classe déclare alors son `@UMLAssociation`, et chacune a raison de le faire — elle décrit ce que son propre code contient.

UML, lui, n'y voit qu'**une** association. Une association binaire est un objet à deux extrémités, dont chacune porte un rôle et une multiplicité, et dont chacune est nommée *par la classe d'en face* : le rôle de l'extrémité `Eleve` est le nom sous lequel `Cours` la désigne. Traduire les deux attributs séparément produisait donc deux traits parallèles là où le lecteur en attend un :

```
Eleve --> "cours 0..*" Cours        % ce que disait Eleve
Cours --> "inscrits 1..*" Eleve     % ce que disait Cours
```

La paire est désormais fusionnée, et le dessin porte le trait unique qu'UML décrit, avec les deux rôles :

```
Eleve "inscrits 1..*" --> "cours 0..*" Cours
```

Où la fusion a lieu, et pourquoi pas ailleurs. Au **rendu**, et non à l'extraction. Le modèle doit continuer de dire fidèlement ce que chaque classe déclare : c'est lui que le processeur d'annotations reconstruit pour vérifier une annotation, et une classe qui déclare une association en déclare une, que l'autre bout en déclare une aussi ou non. La fusion est une décision de *dessin*, prise quand les deux boîtes sont sous les yeux — elle n'a d'ailleurs pas de sens pour une cible hors du diagramme, qui n'a pas de côté à faire valoir.

Ce qui empêche la fusion, et c'est le point délicat. Une extrémité source ne sait porter qu'un rôle, une multiplicité et un qualificatif. Tout ce que le second lien dirait de plus n'aurait nulle part où aller : le losange d'une composition, qui se place à la source du lien qui la déclare et se retrouverait du mauvais côté ; un libellé, qui appartient au trait entier ; une contrainte, un `{ordered}` ou une barre de dérivation, qui qualifient une extrémité que la syntaxe ne sait pas décorer.

La règle est donc étroite : on ne fusionne que si le second lien **n'ajoute rien qu'un rôle et une multiplicité**, et que si le premier n'a pas déjà écrit son propre côté source — ce que l'auteur a écrit l'emporte sur ce que la classe d'en face en dirait. Dans tous les autres cas les deux traits restent, et un avertissement le dit en nommant les deux attributs.

Ce n'est pas de la timidité. Perdre en silence ce que l'auteur a écrit serait le seul vrai défaut ; en dessiner deux traits est simplement moins joli, et le reste réparable en déplaçant les décorations d'un seul côté. Le principe est le même que partout ailleurs dans la bibliothèque : quand elle ne peut pas conclure sans risquer d'effacer une intention, elle dessine ce qui a été écrit et le dit.

7 La Classe d'Association : `@UMLAssociationClass`

7.1 Principe

Identifie une classe spécifique comme étant le porteur des attributs d'une relation entre deux autres classes (souvent utilisée pour modéliser des tables de jointure enrichies).

7.2 Pourquoi c'est utile ?

En Java, il est impossible de lier directement un objet à un lien. Il faut obligatoirement créer une classe intermédiaire. Cette annotation permet de signaler au générateur que cette classe intermédiaire doit être dessinée selon la norme UML stricte : une ligne en pointillés reliant le lien principal à la classe d'association.

7.3 Paramètres

— `source` (Class) : La classe de départ de la relation.
— `target` (Class) : La classe d'arrivée de la relation.
Les deux sont obligatoires : une classe d'association qui ne connaîtrait qu'une extrémité ne qualifierait rien.

Le trait est dessiné par elle. Déclarer la classe d'association suffit : le lien entre les deux extrémités est tracé du même coup. Il n'y a donc pas à écrire en plus une `@UMLAssociation` entre `source` et `target` — ce serait un second trait, et le dessin dirait deux fois la même chose.

Ce qui est refusé. Les deux extrémités doivent porter `@UMLClass`, comme toute extrémité de lien, et la classe annotée aussi — le trait en pointillés rejoindrait sinon une place vide. Aucune des deux extrémités ne peut être la classe d'association elle-même : le lien serait rattaché à la boîte même qui le qualifie. Les deux extrémités sont vérifiées ensemble, si bien qu'un auteur qui s'est trompé sur les deux les voit signalées d'un coup.

En revanche, `source` et `target` peuvent désigner *la même* classe : l'association réflexive existe en UML — deux personnes mariées l'une à l'autre — et la classe qui la qualifie est alors parfaitement licite.

7.4 Exemple

```
@UMLAssociationClass(source = Etudiant.class, target = Cours.class)
@UMLClass
public class Inscription {
    private LocalDate dateInscription;
    private double noteObtenue;
}
```

8 La Multiplicité d'une propriété : `@UMLMultiplicity`

8.1 Principe

Déclare combien d'objets une propriété tient, écrit entre crochets à côté de son nom. Elle se pose sur un attribut — le nombre d'objets tenus — ou sur une méthode — le nombre d'objets rendus.

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class CanonicalMnemonic {
    @UMLMultiplicity("0..*")
    private String deprecatedAliases;   // - deprecatedAliases: String
   [0..*]

    @UMLMultiplicity("0..1")
    private int ucp;                    // - ucp: int [0..1]

    @UMLMultiplicity("0..*")
    public String names() { ... }       // + names(): String [0..*]
}
```

8.2 Pourquoi c'est utile ?

La multiplicité d'un *lien* se déclare déjà sur `@UMLAssociation`, par `targetCardinality`. Mais toutes les propriétés multiples ne sont pas des liens : un ensemble de chaînes, une liste d'entiers ou un champ facultatif n'ont **aucune classe à l'autre bout**, et n'ont donc rien à faire dans une association — laquelle exigerait une `target` figurant sur le diagramme. Cette annotation dit le nombre, et rien que le nombre.

Elle reste par ailleurs le seul moyen de qualifier une **méthode** : `names(): String [0..*]` se lit « rend zéro, une ou plusieurs chaînes », ce qu'aucune association ne saurait exprimer.

8.3 Ce qu'elle ne déduit pas, et ce que le projet peut traduire

L'annotation ne déduit rien, comme partout ailleurs. Mais une partie de la multiplicité **se lit** : `Optional<X>` veut dire « zéro ou un », un `Set<X>` veut dire « zéro ou plusieurs » — le conteneur est la façon dont Java écrit ces multiplicités-là. La règle `multiplicity brackets` du fichier `umlg.rules` (section 15) les transcrit en notation UML pour tout le projet : `aliases : String [0..*]`, `ucp : int [0..1]`. Elle ne dit jamais *plus* que le type — `0..*` et non `1..*`, qu'aucun `Set` ne garantit ; c'est précisément ce que l'annotation, elle, sait dire, et elle l'emporte sur la traduction quand les deux sont présentes. Mesuré sur un projet réel, la règle a remplacé les cinq `@UMLMultiplicity` que le code portait — tous redisaient ce que le type disait déjà.

Au passage du conteneur à l'élément, un type enveloppe redevient primitif : `Optional<Integer>` s'écrit `int [0..1]`, et non `Integer [0..1]` — Java interdit `Optional<int>`, l'enveloppe est une contrainte du langage, pas un choix de l'auteur.

8.4 Paramètres

— `value` (String) : la multiplicité, dans le vocabulaire d'UML — `"1"`, `"0..1"`, `"*"`, `"1..*"`, `"2..5"`. C'est la même grammaire que celle des multiplicités de lien, et c'est la même règle qui la vérifie : il n'y en a qu'une en UML, qu'elle se lise le long d'un trait ou à côté d'un nom.

Le paramètre se nomme `value`, ce qui permet la forme courte `@UMLMultiplicity("0..*")` : le nom de l'annotation dit déjà ce que la valeur est.

8.5 Combinaison avec les autres annotations

— **Avec `@UMLAssociation` sur le même attribut : erreur de compilation**. La multiplicité de cette propriété est déjà donnée par `targetCardinality` ; deux multiplicités sur la même propriété se contrediraient tôt ou tard, et rien ne dirait laquelle fait foi.

— **Avec `@UMLDerived` et `@UMLConstraint`** : sans difficulté, chacune écrivant à sa place — `- /etiquettes: String [1..*] {sans doublon}`.

— **Sur un membre que le diagramme n'affiche pas : avertissement**. Des crochets ne se voient que si le membre qu'ils suivent est dessiné.

9 Les Éléments Dérivés : `@UMLDerived`

9.1 Principe

Indique qu'un attribut, une méthode ou une association est "dérivé", c'est-à-dire qu'il n'est pas stocké directement, mais calculé dynamiquement à partir d'autres données. L'annotation ne s'applique pas à une classe : la notion de classe dérivée n'existe pas en UML.

9.2 Pourquoi c'est utile ?

En UML, un élément dérivé doit être précédé d'un slash (/). Comme ce slash n'est pas une syntaxe Java valide pour nommer une variable, l'annotation signale au générateur d'ajouter ce préfixe visuel lors du rendu PlantUML.

9.3 Paramètres

Aucun.

La barre oblique appartient au **nom** de la propriété, et à lui seul : ni le type, ni la visibilité, ni la multiplicité ne la portent. Sur un lien dessiné en trait, elle précède le rôle cible ; sur une association écrite dans la boîte, le nom de la ligne.

9.4 Exemple

```
@UMLClass(strategy = UMLVisibility.PRIVATE)
public class Personne {
    private LocalDate dateNaissance;

    @UMLDerived
    private int age; // Affiche en UML sous la forme : - /age: int

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetRole = "encours",
                    targetCardinality = "0..*")
    private List<Commande> commandesEnCours; // Trait portant "/encours
    0..*"

    @UMLDerived
    @UMLAssociation(target = Commande.class, targetCardinality = "0..*",
                    notation = UMLNotation.ATTRIBUTE)
    private List<Commande> commandesSoldees; // - /commandesSoldees:
    Commande [0..*]
}
```

10 Les Contraintes Métier : `@UMLConstraint`

10.1 Principe

Permet d'ajouter une règle formelle, exprimée entre accolades `{ }`, applicable à un objet, un attribut ou un lien.

10.2 Pourquoi c'est utile ?

Pour conserver la validité sémantique du modèle, certaines règles métier ne peuvent être exprimées que par des contraintes textuelles (ex : s'assurer qu'un solde est positif, ou marquer un sous-ensemble `{subset}`).

10.3 Paramètres

— `rule` (String) : La règle à afficher, **écrite nue**. Les accolades sont ajoutées par le générateur ; les écrire soi-même les afficherait en double, ce dont le compilateur avertit.

Annotation **répétable**. Si l'élément annoté porte également un `@UMLAssociation`, la contrainte est rattachée au lien et non à l'attribut (voir section 2).

10.4 Où la contrainte est écrite

Trois positions, selon ce que l'annotation vise, pour une seule et même notation :

— **sur un attribut** : à la suite de sa ligne, dans la boîte — `- solde: double {toujours positif}` ;

— **sur un attribut porteur de `@UMLAssociation`** : là où l'association est dessinée — à l'extrémité cible du trait sous `notation = ARROW`, à la fin de la ligne sous `ATTRIBUTE` — aux côtés de la multiplicité et de `{ordered}`. Rien ne distingue alors un `{subset}` écrit par le développeur d'un `{ordered}` engendré, et rien ne doit les distinguer ;

— **sur une classe** : dans une bulle accolée à la boîte. PlantUML n'a pas de notation propre pour une contrainte de classe ; UML l'écrit entre accolades à côté de l'élément, ce qu'une note reproduit exactement.

Une contrainte posée sur un attribut que le diagramme n'affiche pas ne serait écrite nulle part : le compilateur avertit.

10.5 Exemple

```
public class CompteBancaire {

    @UMLConstraint(rule = "solde >= decouverAutorise")
    private double solde; // Affichera {solde >= decouverAutorise} a
   cote de l'attribut
}
```

11 Les Liens Transversaux : `@UMLDependency`

11.1 Principe

Déclare une dépendance vers une autre classe qui n'est pas présente en tant qu'attribut de classe (ex : utilisée localement dans une méthode).

11.2 Pourquoi c'est utile ?

Cette annotation est la seule façon de révéler formellement sur le diagramme qu'une classe en utilise activement une autre, sans pour autant stocker son état de manière prolongée.

11.3 Paramètres

— `target` (Class) : Classe ciblée.

— `label` (String) : Raison du lien (ex : `"utilise"`).

Annotation **répétable** : une classe peut déclarer autant de dépendances que nécessaire.

11.4 Exemple

```
@UMLDependency(target = ServiceBancaire.class, label = "facture via")
public class Panier {
    public void valider() {
        ServiceBancaire sb = new ServiceBancaire();
        sb.payer();
    }
}
```

12 La Documentation Visuelle : `@UMLNote`

12.1 Principe

Génère une bulle de commentaire (note jaune typique de l'UML) rattachée à une classe, un attribut ou une méthode.

12.2 Pourquoi c'est utile ?

Un diagramme explique le "Comment" (structure), la note explique le "Pourquoi". Elle est cruciale pour documenter un algorithme complexe ou avertir les autres développeurs d'une dette technique directement sur le schéma.

12.3 Paramètres

— `text` (String) : Contenu de la note. Obligatoire et non vide — une note sans texte n'est qu'un carré jaune posé au hasard.

— `direction` (Enum) : `UMLDirection` — `UP, DOWN, LEFT, RIGHT`. `NONE`, le défaut, place la bulle **à droite** : c'est le placement d'usage, et celui qui perturbe le moins la lecture verticale de l'héritage.

Annotation **répétable** : plusieurs notes peuvent être rattachées au même élément.

12.4 Ce à quoi une note peut s'accrocher

Une bulle ne se dessine qu'attachée à un élément réellement présent sur le diagramme. Deux cas la laissent donc sans support, et le compilateur avertit dans les deux :

— **le membre visé n'est pas affiché** (masqué par la `strategy`, ou écarté par le filtre anti-bruit) ;

— **l'attribut visé porte `@UMLAssociation` dessinée en trait** (`notation = ARROW`) : il a quitté la boîte pour devenir un trait, et les notes rattachées à un lien sont hors périmètre (section 20). Poser la note sur la classe est alors la seule façon d'en parler.

Sous `notation = ATTRIBUTE`, en revanche, l'association *est* une ligne de la boîte : la note s'y accroche comme à n'importe quel attribut. Elle vise alors le **nom affiché** — le rôle cible s'il est renseigné, le nom du champ sinon.

12.5 Exemple

```
public class MoteurDeRecherche {
    @UMLNote(text = "Algorithme critique, ne pas modifier", direction =
    UMLDirection.RIGHT)
    public void indexer() {
    }
}
```

13 Le Regroupement Logique : `@UMLPackage`

13.1 Principe

Regroupe visuellement plusieurs classes au sein d'un espace nommé (un rectangle de *package* ou de *namespace* en PlantUML).

13.2 Pourquoi c'est utile ?

Sur un projet contenant 40 classes, le diagramme devient illisible. Regrouper les classes par domaine métier (ex : "Sécurité", "Facturation", "Catalogue") permet de diviser la complexité visuelle, sans forcer les développeurs à restructurer leurs vrais dossiers Java.

13.3 Paramètres

— `name` (String) : Nom du groupe. **Obligatoire** — un groupe anonyme ne dirait pas ce qu'il rassemble, et c'est tout ce qu'on lui demande. C'est aussi lui qui fait l'identité du groupe : deux classes déclarant le même `name` entrent dans le même rectangle.
— `color` (String) : Couleur de fond du groupe, écrite comme celle d'une boîte et vérifiée par la même règle.

Le groupe n'est pas le paquetage Java. Il est déclaré, jamais déduit du paquetage où la classe se trouve. C'est délibéré : le découpage qui fait un bon diagramme n'est pas toujours celui qui fait une bonne arborescence de fichiers, et l'un ne doit pas obliger à refaire l'autre. Deux classes de paquetages différents peuvent ainsi figurer dans le même rectangle, et inversement.

Deux couleurs pour un groupe. Un rectangle n'en a qu'une. La première rencontrée est retenue et un avertissement est émis (section 2) — l'auteur a écrit une couleur qu'il ne verra pas, et rien sur le dessin ne le lui dirait. Une couleur déclarée là où le groupe n'en avait pas encore ne concurrence aucun choix : elle est simplement retenue.

Ce que le groupe ne change pas. Les boîtes qu'il entoure sont écrites à l'identique, et les traits continuent de le traverser : une association entre deux groupes reste un trait ordinaire. Un rectangle range les boîtes, il ne les modifie pas.

13.4 Exemple

```
@UMLPackage(name = "Module Paiement", color = "#EEEEEE")
@UMLClass
public class Facture {
}
```

14 Le Correcteur de Mise en Page : `@UMLLayout`

14.1 Principe

Crée un lien invisible (`-[hidden]->`) entre deux classes dans le fichier PlantUML final pour forcer le moteur de rendu à les positionner l'une par rapport à l'autre.

14.2 Pourquoi c'est utile ?

Le moteur PlantUML calcule les positions automatiquement. Parfois, il croise des flèches inutilement ou aligne mal les boites. Cette annotation permet de forcer PlantUML a dessiner comment on veut pouvoir visualiser certaines boites (ex : si on veut que la classe "Voiture" se situe juste au dessus de la classe "moteur" sur le graphique). Attention : C'est une annotation qui peut être dangereuse si utilisé trop souvent.

14.3 Paramètres

— `target` (Class) : Classe cible du lien invisible. Obligatoire, et soumise à la même exigence que toute extrémité : elle doit porter `@UMLClass`.
— `direction` (Enum) : `UMLDirection` — ex : `DOWN` (force la cible à être en dessous). `NONE`, le défaut, pose bien un lien mais ne dit pas de quel côté : il rapproche seulement les deux boîtes, et le compilateur avertit — l'annotation est écrite pour corriger un placement, et celle-là ne corrige presque rien.

Annotation **répétable**, à utiliser avec parcimonie. Une même cible ne peut cependant être placée deux fois : « en haut » puis « en bas » se contrediraient, et le moteur trancherait seul. C'est une erreur de compilation, comme le placement d'une classe par rapport à elle-même.

C'est la seule annotation sans aucun sens UML. Toutes les autres disent quelque chose du modèle. Celle-ci ne dit rien qu'un diagramme puisse signifier — elle corrige un dessin. Quelques-unes redressent une mise en page ; une vingtaine la fige, et le moindre ajout de classe oblige alors à toutes les reprendre.

14.4 Exemple

```
@UMLLayout(target = BaseDeDonnees.class, direction = UMLDirection.DOWN)
@UMLClass
public class InterfaceUtilisateur {
}
```

15 Les règles du projet : le fichier `umlg.rules`

15.1 Principe

Certaines décisions ne portent pas sur un élément mais sur le projet entier : « dans ce diagramme, un `ImmutableList` s'affiche `List` ». Elles se déclarent dans un fichier facultatif, `umlg.rules`, posé **à la racine du répertoire analysé**. Pour un projet Maven ordinaire, cela signifie `src/main/resources/umlg.rules`, que la construction recopie dans `target/classes`.

15.2 Pourquoi un fichier, et non une annotation ?

Une annotation dit une chose sur un élément : cette classe-ci est une boîte, cet attribut-là est un lien. C'est le bon outil pour ce qui se décide au cas par cas, et le mauvais pour ce qui vaut partout — écrire la même directive sur chacun des deux cents attributs d'un projet est un travail que personne ne fait, et que personne ne tient à jour. Le fichier sert exactement le principe de sobriété énoncé en introduction : une règle générale s'écrit une fois, à un endroit, et ne coûte plus rien ensuite.

15.3 Forme du fichier

Une règle par ligne. Les lignes vides sont ignorées, ainsi que les commentaires, ouverts par `#`. Une règle commence par un mot-clé qui dit sa nature. Le vocabulaire est **fermé** : un mot inconnu arrête la génération plutôt que d'être ignoré.

15.4 Le vocabulaire complet

Règle	Ce qu'elle fait
Ce que les boîtes montrent	
`strategy public`	La stratégie de visibilité des classes qui ne déclarent pas la leur. Valeurs : `none`, `public`, `protected`, `package`, `private`.
`hide accessors`	Écarte `getX()`, `isX()`, `setX(v)`, à la convention JavaBean stricte.
`hide object-methods`	Écarte `toString()`, `hashCode()`, `equals(Object)`.
`hide record-accessors`	Écarte les accesseurs engendrés d'un *record*.
`hide enum-methods`	Écarte `values()` et `valueOf(String)`.
`hide method get*, with*`	Écarte les méthodes dont le nom suit l'un des motifs.
`hide field log, *Cache`	Idem pour les attributs — le seul moyen d'en filtrer.
`hide field Representation.string`	Un motif **qualifié** — `Classe.membre` — désigne un membre nommément ; lui seul l'emporte sur `@UMLInclude` (section 15.7).
`hide return void`	Efface le type rendu : `valider()` et non `valider(): void`.
`hide parameter names`	Tait les noms des paramètres : `from(KeyboardMap)` et non `from(keyboardMap : KeyboardMap)`. La forme des diagrammes tenus à la main, qui divise la largeur des boîtes bavardes.
`type ImmutableList -> List`	Renomme un type partout où il paraît dans une boîte, y compris à l'intérieur des chevrons.
`hide visibility`	Tait les symboles `+`, `-`, `#`, `~` devant les membres. UML les demande, mais beaucoup de diagrammes tenus à la main les omettent, parce qu'ils montrent une structure plutôt qu'une API.
`order declaration`	Range les membres dans l'ordre du code, relu dans le fichier `.class`, plutôt que par alphabet. Aussi : `order alphabetical`, le défaut.
`multiplicity brackets`	Traduit la forme des conteneurs en multiplicités : `Set<X>` s'écrit `X [0..*]`, `Optional<X>` s'écrit `X [0..1]` — l'élément déballé, primitif compris. `@UMLMultiplicity` l'emporte (section 8).
Ce que le dessin d'ensemble montre	
`title Mon architecture`	Titre écrit en tête de la vue intégrale.
`direction left-to-right`	Sens de lecture de la vue intégrale. Aussi : `top-to-bottom`.
`show empty members`	Rétablit les compartiments vides, masqués par défaut.
`group package com.x -> "Modèle"`	Range tout un paquetage dans un rectangle nommé.
Les vues (section 17)	
`view "Nom" = A, B, package c.d`	Déclare une vue et ce qu'elle retient, dans l'ordre.
`view "Nom" = package c.d - A`	La même, moins ce qui suit le tiret.

15.5 Un fichier complet

```
# ---- Le dessin d'ensemble ----
title Architecture du module Facturation
direction left-to-right

# ---- Ce que les boites montrent ----
strategy public
hide accessors
hide object-methods
hide method with*, visit*
hide field log, LOGGER
hide return void
hide visibility
hide parameter names
order declaration
multiplicity brackets
type ImmutableList -> List

# ---- Le rangement ----
group package com.exemple.modele -> "Modele"

# ---- Les vues ----
view "Le domaine" = package com.exemple.modele
view "Facturation" = Facture, LigneFacture, Client
view "Facturation" direction left-to-right
view "Autour de Facture" = around Facture

views by package
output replace
```

15.6 `strategy` : la visibilité de tout le projet

Une règle `strategy` donne la stratégie que prennent les classes qui n'en déclarent pas — c'est-à-dire celles dont le `strategy` vaut `PROJECT`, qui est le défaut de `@UMLClass`. C'est la règle qui rend vraie la promesse de l'introduction : sans elle, obtenir un diagramme peuplé demandait d'écrire `@UMLClass(strategy = ...)` sur chacune des classes du projet. Une classe qui déclare la sienne l'emporte, dans les deux sens : `PRIVATE` pour en montrer davantage, `NONE` pour la ramener à sa boîte nue.

Sans aucune ligne `strategy`, le défaut ne dessine aucun membre, et les boîtes sortent vides — la surprise type d'un premier essai, que la génération **signale** désormais en nommant les remèdes (section 17.9). Un `strategy none` **écrit** produit le même dessin sans le signalement : une ligne écrite est un choix, pas un oubli.

15.7 `hide` : ce que le projet tient pour du bruit

Voir la section 3.3 pour le principe — rien n'est masqué sans demande. Quatre familles se nomment d'un seul mot ; elles sont indépendantes, et demander l'une ne demande pas les autres.

Ce qu'aucune famille ne reconnaît se désigne par un **motif de nom**. L'étoile y remplace n'importe quelle suite de caractères ; tout le reste est pris au pied de la lettre, et la casse compte. Plusieurs motifs se séparent par des virgules. C'est ainsi qu'on écarte les conventions propres à un projet — les `withX` d'un constructeur fluide, les `visitX` d'un

visiteur — et c'est le seul moyen de filtrer des **attributs**, qu'aucune forme ne trahit comme la convention JavaBean trahit un accesseur.

`@UMLInclude` l'emporte sur une règle générale : le cas particulier, écrit sur le membre, bat le réglage qui vaut partout.

Le motif qualifié, seule exception. Un motif contenant un point — `Representation.string`, ou `Representation.*` pour toute la boîte — ne désigne plus une famille de noms mais **un membre d'une classe précise**. Cette désignation nommément est aussi individuelle que l'annotation elle-même, et c'est ce qui justifie son pouvoir : elle l'emporte *aussi* sur `@UMLInclude`. Le cas d'usage est celui des boîtes déclarées (section 18) : quand deux sous-classes déclarées redisent les champs de la classe réelle, la boîte parente doit pouvoir se taire — et retirer les `@UMLInclude` du code n'est pas une réponse, ils servent au reste du diagramme. Le motif qualifié se compare à `Classe.membre`, le motif ordinaire au nom seul, et les deux ne se croisent pas : `s*` ne gagne aucun pouvoir sur `Representation.string`.

15.8 `type` et `hide return` : les types affichés

Une règle `type` remplace un nom de type par un autre, **partout où ce type paraît dans une boîte** : attributs, types de retour, paramètres de méthode. Le nom de gauche s'écrit au choix simple (`ImmutableList`) ou complet (`com.google.common.collect.ImmutableList`) ; le nom complet l'emporte sur le nom simple, ce qui permet de viser une classe précise quand deux paquetages en ont une homonyme.

Une règle `hide return` efface le type rendu par les méthodes qui le rendent. UML tient ce type pour facultatif, et l'omettre veut dire « ne rend rien ». La règle vaut pour le **seul retour**, et c'est délibéré : un attribut ou un paramètre privé de son type laisserait une ligne incompréhensible, là où une méthode sans type de retour dit quelque chose de précis.

15.9 `title`, `direction`, `show empty members` : le dessin d'ensemble

Ces trois réglages étaient autrefois portés par une annotation, `@UMLDiagram`, posée sur un `package-info.java`. Elle a été retirée, et l'arbitrage mérite d'être dit : ils ne disent rien d'une classe, ils valent pour le dessin entier — exactement ce que ce fichier existe pour porter. Les vues ont achevé de la vider de son sens, chacune déclarant désormais son propre titre et son propre sens de lecture. Il ne lui restait que la vue intégrale, pour trois lignes que voici.

`show empty members` demande à voir les compartiments d'attributs et de méthodes restés vides, masqués par défaut. Ce défaut n'est pas neutre : sans stratégie déclarée, une boîte n'a aucun membre, et ne pas masquer dessinerait deux rectangles vides sous chaque nom.

15.10 `group package` : ranger un paquetage

Une règle `group package` place toutes les classes d'un paquetage Java dans un même rectangle, ce qui remplace autant de `@UMLPackage` qu'il y a de classes. La correspondance est **exacte** : `com.exemple.modele` ne range pas les classes de

`com.exemple.modele.detail`. Deux règles qui se recouvriraient demanderaient une règle de priorité, et un sous-paquetage ajouté entrerait en silence dans un rectangle que personne n'a désigné.

Le rectangle ainsi obtenu n'a pas de couleur ; `@UMLPackage` reste le moyen d'en donner une. Une classe qui porte l'annotation l'emporte sur la règle.

15.11 Ce que les règles ne touchent pas

Le **nom d'une boîte** et la **classe visée par une** `@UMLAssociation`. Ces deux-là ne sont pas lus dans le code mais écrits dans une annotation — renommer une boîte, c'est `@UMLClass(name = ...)`. Une règle qui les atteindrait ferait diverger le type écrit dans une boîte du nom de la boîte d'à côté, sans que rien ne le signale.

Le nom d'un **membre** suit le même partage : il se change côté code, par `@UMLInclude(name = ...)`, jamais par une règle — c'est une décision sur un élément, écrite là où l'élément est écrit.

15.12 Ce qui est refusé

Le fichier est écrit à la main, sans compilateur pour le relire : chaque refus nomme donc le fichier, le numéro de ligne et la forme attendue. Un fichier fautif **arrête la génération** plutôt que d'être appliqué à moitié — il a été écrit pour être suivi, l'ignorer produirait un diagramme qui n'est pas celui qu'on a demandé.

Situation	Raison
Fichier qui n'est pas en UTF-8	Le premier accent d'un commentaire enregistré en CP-1252 rendait la lecture illisible derrière une exception générique ; le refus nomme désormais l'encodage attendu.
Mot-clé inconnu (`couleur Client -> rouge`)	Le vocabulaire du fichier est fermé ; le message rappelle les mots-clés connus.
Stratégie inconnue (`strategy publique`), ou `strategy project`	Les valeurs sont celles de l'énuméré ; `project` est ce qu'une classe écrit pour renvoyer à cette ligne, qui ne peut pas se renvoyer à elle-même.
Deux lignes `strategy`	La seconde l'emporterait en silence.
Famille de bruit inconnue, ou demandée deux fois	Le vocabulaire est fermé ; le message rappelle les quatre familles.
Motif vide, ou réduit à `*`	Une virgule orpheline, le plus souvent ; `*` masquerait tout le compartiment.
Deux `group package` sur le même paquetage	Le paquetage ne peut appartenir qu'à un rectangle.
Deux lignes `title`, ou `title` sans texte	La seconde l'emporterait en silence ; un titre vide ne s'écrit pas.
`direction` ou `output` de valeur inconnue	Le vocabulaire est fermé ; le message rappelle les valeurs admises.
`view` sans nom entre guillemets, ou sans `=`	La vue ne dit pas comment elle s'appelle, ou ce qu'elle contient.
Vue déclarée deux fois, ou `direction` avant son contenu	La seconde l'emporterait ; le sens de lecture d'une vue inexistante ne s'applique à rien.
Sélecteur ne désignant aucune classe	Une classe renommée laisserait un nom mort, et le diagramme se dessinerait amputé sans que rien ne le signale.
Flèche `->` absente, ou l'une des deux moitiés vide	La règle ne dit pas ce qu'elle remplace, ou par quoi.
`hide` sans `return` (`hide void`)	La règle ne dit pas où elle s'applique ; le message rappelle toutes les formes de `hide`.
`order` de valeur inconnue, ou deux lignes `order`	Deux valeurs seulement, `declaration` et `alphabetical` ; deux lignes parleraient du même réglage, même quand la seconde redit le défaut.
Deux `hide parameter names`	La seconde ne demanderait rien de plus ; un fichier qui se répète est un fichier que personne ne relit.
Deux règles sur le même type	La seconde l'emporterait en silence, et l'auteur ignorerait laquelle s'applique.
Espace au milieu d'un nom de type	Trahit une flèche oubliée ou un mot en trop.

Un cas échappe à ce tableau parce qu'il ne peut pas être un refus : une règle de nommage dont le nom ne correspond à rien — `type ImutableList -> List`, `hide field LOGGR` — est syntaxiquement irréprochable, et un motif a le droit de ne viser personne aujourd'hui. Elle n'est donc pas refusée, mais elle n'est plus muette : la génération relève en fin de scan les lignes restées sans effet et les signale en avertissement (section 17.9).

15.13 Une extension prévue

Le fichier est fait pour grandir : d'autres natures de règles viendront s'ajouter à celles-ci — ce qu'on masque partout, ce qu'on abrège, ce qu'on place. Le lecteur du fichier est écrit pour cela : ajouter une règle consiste à reconnaître un mot-clé de plus, et un mot-clé inconnu est refusé plutôt qu'ignoré, ce qui garantit qu'un fichier écrit pour une version ultérieure ne sera jamais appliqué à moitié par une version antérieure.

16 L'apparence du diagramme : `UmlStyle`

16.1 Principe

L'habillage du dessin — couleurs des boîtes, épaisseur et teinte des traits, police — ne se déclare ni sur un élément ni dans le fichier de règles, mais **à l'appel du générateur**, par l'énuméré `UmlStyle`. C'est le troisième et dernier endroit où quelque chose se décide, et le seul qui ne touche pas au modèle : les mêmes boîtes, les mêmes traits, les mêmes membres, vêtus autrement.

```
UMLGenerator.builder()
    .sourceDirectory("target/classes")
    .outputFile("architecture.svg")
    .style(UmlStyle.UMLG)    // facultatif ; PLANTUML par defaut
    .build()
    .generate();
```

16.2 Pourquoi ce n'est pas une annotation

Une annotation dit une chose sur un élément, et le fichier `umlg.rules` dit une chose sur le modèle du projet — qu'un `ImmutableList` s'affiche `List`. L'apparence ne dit rien, ni de l'un ni de l'autre : deux diagrammes engendrés dans les deux styles portent exactement la même information. Elle relève de qui *regarde* le diagramme, non de qui l'écrit, et peut changer d'une génération à l'autre sans qu'une ligne de code annoté bouge. La placer dans le code source obligerait à recompiler pour changer de couleur.

Cela n'entre pas en contradiction avec les directives brutes écartées en section 20. Le vocabulaire est ici **fermé** : deux valeurs, et aucun accès aux `skinparam` de PlantUML. On choisit entre deux habillages tenus par la bibliothèque, on n'en compose pas un troisième.

16.3 Les deux valeurs

Valeur	Ce qui est dessiné	Quand la choisir
`PLANTUML` (défaut)	Ce que PlantUML dessine quand on ne lui dit rien : boîtes grises, pastille `C` ou `I` accolée au nom, traits par défaut.	Le cas nominal. Un utilisateur qui n'a rien demandé retrouve le rendu qu'il reconnaît de tous les diagrammes qu'il a déjà vus.
`UMLG`	Bandeau bleu pâle sur fond blanc cassé, bords bleu foncé, coins arrondis, pas d'ombre, nom des liens en italique coloré, pastille masquée.	À l'impression, ou sur un fond clair : plus sobre, au prix d'un repère visuel en moins.

Le défaut est `PLANTUML` par le même principe que partout ailleurs : ne rien imposer est un meilleur point de départ qu'imposer un goût.

16.4 Ce qui échappe au choix

Deux réglages sont posés dans les deux cas, parce qu'ils touchent au sens et non à l'apparence :

— `classAttributeIconSize 0` : les visibilités s'écrivent `+`, `-`, `#`, `~` — la notation d'UML — et non en pastilles colorées propres à PlantUML ;

— l'**espacement**, calculé sur la plus longue étiquette du diagramme, sans quoi les étiquettes de lien se chevaucheraient.

Un troisième leur ressemble sans leur appartenir : le masquage des **compartiments vides**, que les deux apparences écrivent pareillement mais que la ligne `show empty members` du fichier de règles peut lever (section 15.4). Ce n'est donc pas un invariant du rendu, c'est un réglage du diagramme sur lequel le style n'a pas d'avis — les deux apparences le suivent, elles ne le décident pas.

17 Les vues : plusieurs diagrammes pour un projet

17.1 Principe

Un projet de trente classes donne un dessin que personne ne lit. Ce qu'on lit, ce sont quelques diagrammes de cinq à dix boîtes, chacun répondant à une question — « comment se fait la facturation ? », « que dois-je regarder pour comprendre cette classe-ci ? ». La bibliothèque produit donc, en une exécution, **un dossier de diagrammes** plutôt qu'une image :

```
diagrammes/
|-- projet.svg          <- la vue integrale, toujours produite
`-- vues/
    |-- le-domaine.svg
    |-- autour-de-compte.svg
    `-- com.banque.modele.svg
```

La **vue intégrale** est toujours écrite, à la racine : c'est l'inventaire du projet, celui qu'on ouvre pour savoir ce qui existe. Elle porte le titre et le sens de lecture déclarés par les lignes `title` et `direction` du fichier de règles. Les autres vues vivent dans un sous-dossier ; chacune porte son propre titre — son nom — et son propre sens de lecture.

17.2 Déclarer une vue

```
# Ce que la vue retient, dans l'ordre ou on veut le lire.
view "Le domaine" = package com.banque.modele
view "Facturation" = Facture, LigneFacture, Client
view "Autour de Compte" = around Compte

# Ce qu'elle retire, apres un tiret entoure d'espaces.
view "Le domaine sans les lecteurs" = package com.banque -
    XkbSymbolsReader

# Son sens de lecture, sur une ligne a part.
view "Facturation" direction left-to-right
```

Trois façons de désigner des classes, et trois seulement :

Sélecteur	Ce qu'il retient
`Facture`	Une classe, par son nom simple ou complet. C'est la forme qui reproduit un diagramme tenu à la main : on choisit, une par une.
`package com.exemple`	Toutes les classes du paquetage, correspondance exacte — un sous-paquetage n'entre pas.
`around Facture`	La classe et ses voisines immédiates : celles qu'elle touche, celles qui la touchent. Le rayon est de un, délibérément : à deux, le voisinage d'une classe centrale redevient le projet entier.

Un voisinage peut être vide. Les voisins d'`around` sont les extrémités des **relations déclarées** — `@UMLAssociation`, `@UMLDependency`, héritages entre classes annotées — et non les types que le code manipule. Une classe très entourée dans le code peut donc n'avoir aucun voisin déclaré, et la vue se réduit alors à une boîte seule. Ce n'est pas une faute — le diagramme est exact — mais c'est presque toujours une surprise, et sa cause ne se lit pas sur le dessin : la génération **avertit**, en expliquant d'où viennent les voisins et en nommant les deux remèdes — déclarer une relation, ou nommer explicitement les classes de la vue.

Un sélecteur qui ne désigne aucune classe **arrête la génération**. Le fichier n'a pas de compilateur pour le relire : une classe renommée y laisserait un nom mort, et le diagramme se dessinerait amputé sans que personne ne le sache.

17.3 L'ordre d'écriture est l'ordre de lecture

Les classes d'une vue sont dessinées dans l'ordre où elles sont écrites. Ce n'est pas un détail de présentation, c'est le seul moyen d'exprimer quelque chose que le code ne contient pas.

Le moteur de rendu place déjà correctement ce qui est relié : une association pose sa source au-dessus de sa cible, et la mise en page suit les traits. Ce qu'il ne peut pas deviner, c'est l'ordre de deux boîtes que *rien* ne relie — il les pose alors côte à côte, au hasard de son algorithme. Or cet ordre-là est celui de l'exposé qui accompagne le dessin : on présente d'abord la carte, puis le lecteur, puis le convertisseur.

La bibliothèque enchaîne donc par un lien invisible les **blocs** de la vue, dans l'ordre où leur première boîte est écrite. Un bloc est un groupe de boîtes que les traits relient, directement ou de proche en proche.

Pourquoi des blocs, et non des boîtes. C'est le point délicat. Enchaîner deux à deux les boîtes consécutives qu'aucun trait ne joint semble équivalent, et ne l'est pas : trois classes filles d'un même parent ne sont pas reliées *entre elles*, et se retrouveraient empilées l'une sous l'autre, alors que le moteur les dispose en éventail sous leur parent — la forme même que l'héritage doit avoir. Ce qu'il faut ordonner, ce sont les groupes qui se tiennent, pas les boîtes.

Ainsi, une vue retenant `KeysymEntry` et ses trois filles, puis `KeyboardMap`, puis `XkbSymbolsReader`, reçoit exactement deux liens invisibles — `KeysymEntry` vers `KeyboardMap`, puis `KeyboardMap` vers `XkbSymbolsReader` — et pas un de plus. Le reste, le moteur s'en charge, et il le fait mieux que nous.

C'est ce qui rend `@UMLLayout` presque inutile ; elle reste pour le cas ponctuel, une boîte mal placée dans une vue par ailleurs correcte.

L'ordre quand rien n'est écrit. Un sélecteur `package` ou une vue calculée ne portent aucun ordre : leurs boîtes sont alors rangées par nom complet. Ce n'est pas un choix esthétique mais une exigence de reproductibilité — le parcours du système de fichiers, lui, ne promet aucun ordre, et deux machines analysant le même code produiraient sinon deux dessins différents. C'est la même raison qui fait trier les membres d'une boîte.

17.4 Les vues calculées

```
views by package
```

Une vue par paquetage Java, sans rien déclarer. C'est le découpage que suivent la plupart des diagrammes tenus à la main, parce qu'un paquetage rassemble déjà ce qui se comprend ensemble. Il ne dispense pas de déclarer des vues : celles-ci retiennent *tout* le paquetage, là où un auteur en écarterait les classes techniques.

Les boîtes **déclarées** dans le fichier de règles (section 18) n'y entrent pas : elles n'ont pas de paquetage — leur nom qualifié est leur nom simple — et les ranger dans un paquetage « (défaut) » fabriquait une vue entière que personne n'avait demandée, peuplée de boîtes orphelines dont le parent réel, resté dans son vrai paquetage, sortait de la vue avec sa flèche d'héritage. Les vues par paquetage décrivent le code ; ce que le code ne porte pas reste sur la vue intégrale et sur les vues qui le nomment.

17.5 Les liens qui sortent d'une vue

Une relation dont l'autre bout n'est pas dans la vue **disparaît, sans un mot**. C'est le même arbitrage que pour un héritage dont le parent n'est pas annoté : restreindre la vue *est* la demande de l'auteur, il n'y a donc aucun écart à lui signaler. Le contraire — dessiner la boîte manquante pour sauver le trait — ferait déborder chaque vue de proche en proche, jusqu'à ce qu'elles se ressemblent toutes.

17.6 Passer d'une vue à l'autre

Un dossier de onze diagrammes n'est qu'une collection d'images tant qu'on ne peut pas passer de l'une à l'autre. Chaque boîte porte donc un **lien cliquable** vers la vue qui la détaille : depuis la vue intégrale, cliquer sur une classe ouvre le diagramme où elle est expliquée. Le dossier devient une documentation qui se parcourt.

La cible est la première vue déclarée qui retient cette boîte ; une vue ne renvoie jamais vers elle-même, un tel lien ne menant nulle part. Les chemins sont relatifs au fichier courant, si bien que le dossier reste déplaçable.

17.7 Quand le dessin devient illisible

Un diagramme peut être juste et impossible à lire. Une vue dont les boîtes s'alignent sur une seule rangée donne un ruban de plusieurs milliers de pixels de large sur quelques centaines de haut : imprimé sur une page, chaque boîte y fait deux millimètres. Rien n'a échoué, la source est correcte, et l'auteur ne le découvre qu'en ouvrant le fichier.

Deux garde-fous.

L'enchaînement s'arrête à trois blocs. Ordonner deux ou trois groupes dit une intention ; en ordonner cinq construit une file indienne que le moteur déroule sur toute la largeur. Au-delà de trois, la vue n'est donc plus contrainte du tout : le moteur compose mieux que l'ordre écrit, et mieux vaut le laisser faire que d'imposer une lecture qui rendrait le dessin inutilisable. Sur un cas réel, cinq blocs donnaient 3570 pixels de large pour 289 de haut ; les mêmes boîtes sans contrainte tenaient dans 1578 sur 824.

Les proportions sont mesurées. La bibliothèque relit les dimensions de ce qu'elle vient d'écrire, et **avertit** lorsque le rapport entre la plus grande et la plus petite dépasse quatre pour un, sur une image assez grande pour que cela compte. Elle ne corrige rien : les causes sont plusieurs — un sens de lecture qui ne convient pas à cette vue-ci, des boîtes trop larges, une vue qui retient trop de classes — et le choix entre elles appartient à l'auteur. Le message nomme la vue et rappelle les remèdes. La mesure vaut pour les **deux modes de sortie**, le dossier de vues comme le fichier unique — lequel est précisément le mode de qui n'a rien configuré, celui où le ruban surprenait le plus.

17.8 Ce que la génération dit d'elle-même

Un compte rendu, en deux lignes :

```
12 diagram(s) written to /chemin/diagrammes
  21 class(es) drawn, 23 ignored (no @UMLClass)
```

Le second chiffre est celui que personne n'avait. Une classe sans `@UMLClass` est hors du diagramme, et c'est voulu — le périmètre est choisi. Mais rien ne le disait : une classe volontairement écartée et une classe qu'on a oublié d'annoter se ressemblent trait pour trait, c'est-à-dire qu'elles sont absentes toutes les deux. Le compte permet de faire la différence ; sous une douzaine, elles sont même nommées.

La liste complète est écrite, pas seulement comptée. Au-delà d'une douzaine, le compte rendu s'arrête de nommer — la liste noierait le reste — mais c'est précisément le cas qui a motivé la fonctionnalité : sur le projet de référence, quinze classes sur trente et une manquaient au dessin, soit trop pour l'écran. La génération écrit donc `ignored-classes.txt` à la racine de la sortie, avec **toutes** les classes ignorées, une par ligne ; le compte rendu y renvoie, et un fichier périmé d'une génération précédente est retiré. Le fichier est écrit dans les deux modes : en mode fichier unique, l'utilisateur lisait « 37 ignored » sans aucun moyen de savoir lesquelles.

Le compte rendu est rendu, non imprimé. `generate()` rend le `RapportDeGeneration` et n'écrit nulle part : choisir un canal n'appartient pas à une bibliothèque. La ligne de commande l'écrit sur la sortie standard, le plugin Maven le passe à son log, un appelant programmatique peut échouer sur « quinze classes ignorées » sans relire quoi que ce soit. Il l'imprimait en plus de le rendre, et le plugin relayait alors les mêmes lignes : l'utilisateur d'un build lisait tout en double, et la ligne qui renvoie au fichier des ignorées — écrite à part, donc absente du rapport — se perdait au passage.

Les **avertissements** suivent le même principe. Ils partaient tous sur la sortie d'erreur, écrits là où ils naissent ; sous Maven ils s'y écrivaient tels quels, sans préfixe `[WARNING]`, hors du log et absents du récapitulatif de fin — le message le plus utile de l'outil était celui qu'on voyait le moins. L'appelant peut désormais leur donner un canal, et le plugin leur donne celui de Maven.

La source PlantUML est conservée. Chaque image est accompagnée de sa source, sous le même nom et l'extension `.puml` : `projet.svg` et `projet.puml`. Sans elle, la source engendrée était irrécupérable — pas de retouche possible pour un essai, et le diff entre deux générations, seule façon de voir ce qu'un changement de code a changé au dessin, se faisait sur du SVG illisible. Les `.puml` suivent le sort de leurs images : `output replace` les efface avec elles.

En mode fichier unique, la destination est nommée même quand le chemin n'a pas de dossier : `written to architecture.svg`, et non le `written to null` qu'un chemin d'un seul segment produisait — le tout premier message que lisait un utilisateur n'ayant rien configuré.

17.9 Ce que la génération signale

Au compte rendu s'ajoutent des **avertissements**, émis pendant le scan. Ils partagent une même définition : le diagramme est *exact*, mais il dit moins que ce que son auteur croit avoir demandé, et rien sur le dessin n'en montrerait la cause. Et une même retenue : chacun se tait dès que le silence est juste — un choix écrit n'est jamais signalé.

Situation	Ce que l'auteur découvrirait sinon sur l'image
Des boîtes sans membre, alors qu'aucune `strategy` n'est déclarée nulle part	Le défaut ne dessine aucun membre : la surprise type d'un premier essai. Le message nomme les boîtes et les trois remèdes — `strategy public` dans le fichier, `@UMLClass(strategy = ...)`, ou `@UMLInclude` membre par membre. Il se tait devant un `strategy none` **écrit** — une ligne écrite est un choix — et devant une classe sans aucun membre, qui resterait vide sous toute stratégie.
Aucun paramètre d'aucune méthode ne porte de nom	Le projet a été compilé sans `-parameters` (section 2.2) : les signatures se réduisent aux types sans qu'on l'ait demandé. Il se tait si un seul nom existe, ou si `hide parameter names` l'a demandé.
Des `.class` que le chargeur n'a pas su lier	Une dépendance `provided` ou `optional` absente du classpath transmis, ou une compilation partielle. Ces classes sont écartées et nommées plutôt que de faire échouer le scan entier — il suffisait d'une classe dont le diagramme ne voulait pas pour rendre la génération impossible. L'avertissement dit aussi ce qu'on *ne peut pas* savoir : l'annotation ne se lit que sur une classe chargée, donc rien ne dit si elles portaient `@UMLClass`, et le diagramme est peut-être incomplet.
Deux boîtes portant le même nom affiché	Deux classes homonymes de paquetages différents. Les alias qui les désignent restent distincts — chaque trait aboutit à la bonne boîte — mais l'alias ne se lit pas sur le dessin : le lecteur voit deux rectangles identiques. Rien n'est renommé d'autorité, `@UMLClass(name = ...)` est là pour que l'auteur tranche.
Une vue `around` sans voisin	La vue montre la classe seule : les voisins viennent des relations déclarées, pas des types du code (section 17).
Une règle de nommage restée lettre morte	Un `type`, `hide return`, `hide method`, `hide field` ou `group package` qui n'a correspondu à rien pendant le scan. Une classe renommée ou une faute de frappe laisse une ligne morte, et le no-op était jusqu'ici parfait : le diagramme se dessinait, exact, sans montrer la ligne restée sans effet. Les vues, elles, échouent bruyamment sur un nom introuvable ; les règles de nommage ne le peuvent pas — un motif a le droit de ne viser personne — d'où un avertissement, qui cite chaque ligne telle qu'elle a été écrite.
Une vue aux proportions illisibles	Le ruban de quelques milliers de pixels sur quelques centaines, décrit ci-dessus (« Quand le dessin devient illisible »).

Ce qui l'arrête, en revanche. Un avertissement dit « le diagramme est exact mais dit moins que tu ne crois » ; une erreur dit qu'aucun diagramme ne peut être produit. Arrêtent donc la génération : une annotation contradictoire, une couleur que PlantUML ne connaît pas, un fichier de règles fautif ou qui n'est pas en UTF-8, une vue nommant une classe absente du diagramme, un répertoire où aucune classe ne porte `@UMLClass` — le message nomme alors les classes lues, puisque le scan vient de les lire.

S'y ajoute un cas qui n'est pas une faute de l'auteur : **Graphviz introuvable**. Un diagramme de classes est mis en page par Graphviz, que PlantUML appelle en sous-processus. Quand il manque, PlantUML ne lève rien et ne rapporte aucune erreur de diagramme : il dessine une image portant « Cannot find Graphviz », que ni sa taille ni son format ne distinguent d'un dessin réussi, et le compte rendu annonçait un succès. La génération vérifie donc que le moteur est en état avant de dessiner, une fois par exécution, et dit quoi installer. Le principe est le même que pour la source elle-même, dont l'acceptation par PlantUML est vérifiée avant l'écriture : une image d'erreur livrée comme un diagramme est le pire des résultats, puisque rien ne la dénonce.

17.10 Ce que devient la génération précédente

```
output replace   # le defaut
output keep
```

Sous `replace`, les diagrammes précédents sont effacés : le dossier décrit toujours l'état courant du code, et une vue retirée du fichier ne laisse pas derrière elle une image que plus rien ne met à jour — laquelle ment d'autant mieux qu'elle a l'air d'être à sa place. Seuls les fichiers que la bibliothèque écrit sont retirés : un `LISEZ-MOI` posé dans le dossier y reste.

Sous `keep`, rien n'est effacé : la génération est écrite dans un dossier voisin numéroté — `diagrammes-2`, `diagrammes-3` —, ce qui permet de comparer avant et après une modification du code.

17.11 Lancer la génération

Trois façons, selon le contexte.

Depuis Maven , ce qui est la forme à préférer : un **plugin dédié**, `umlg-maven-plugin`, dont les défauts disent l'usage — `target/classes` analysé (avec le fichier `umlg.rules` que la construction y recopie), `diagrammes/` écrit à la racine du projet.

```
<plugin>
    <groupId>io.github.oliviercailloux</groupId>
    <artifactId>umlg-maven-plugin</artifactId>
    <version>1.0-SNAPSHOT</version>
    <executions>
        <execution>
            <goals><goal>generate</goal></goals>
        </execution>
    </executions>
</plugin>
```

puis `mvn compile umlg:generate`. Chaque réglage se surcharge par propriété : `-Dumlg.outputFile=architecture.svg` (un seul fichier), `-Dumlg.format=png`, `-Dumlg.outputDirectory=...`, `-Dumlg.skip=true`. Les chemins relatifs s'entendent

depuis la racine du module, et non depuis le répertoire d'où Maven a été lancé : dans un réacteur, tous les modules écriraient sinon le même fichier. Le goal `umlg:help` énumère ces réglages.

Deux exigences pèsent sur le `pom` du projet analysé, et l'une est un piège. Il faut `maven.compiler.parameters` pour que les signatures portent le nom de leurs paramètres — mais il faut aussi **épingler la version** de `maven-compiler-plugin` : sans elle, Maven résout la 3.1, publiée en 2013 et antérieure au paramètre `parameters`, qui est alors ignoré en silence. La génération signale des signatures anonymes, et conseille d'ajouter ce que l'auteur a déjà écrit. Il faut enfin `showWarnings`, sans quoi Maven passe `-nowarn` à `javac` et fait taire tout ce que le processeur d'annotations a à dire, ses erreurs mises à part.

Consommer la bibliothèque depuis un module. Les trois paquetages exportés rendent des types Guava dans leurs signatures — `RapportDeGeneration.classesIgnorees()` une `ImmutableList`, `UMLGenerator.couleursConnues()` une `ImmutableSortedSet`. Le `module-info` les rend donc lisibles par ses clients (`requires transitive`) : exporter un type sans exporter de quoi le lire laisserait un module client compiler l'appel et échouer sur le type de retour.

Ce plugin remplace l'usage antérieur d'`exec-maven-plugin`, qui exigeait une `mainClass` et deux **arguments positionnels** dont rien ne disait le sens ni l'ordre — le projet de référence en avait accumulé deux déclarations concurrentes, que Maven signalait. Le plugin transmet de surcroît à la bibliothèque le **classpath d'exécution** du projet : la réflexion ne s'arrête pas au répertoire des classes — charger une classe résout son parent, lire un attribut résout son type, qui vivent dans les dépendances du projet — et `exec:java` n'apportait ce classpath que par accident de son mode d'exécution.

```
UMLGenerator <repertoire-analyse> [fichier.svg|dossier] [png|svg]
```

Le second argument dit à lui seul ce qu'on veut : un nom finissant par `.svg` ou `.png` demande ce seul fichier, tout autre nom demande un dossier de vues. Deviner ainsi évite une option de plus, et se lit sans documentation — personne n'appelle « diagrammes » un fichier, ni « architecture.svg » un répertoire. Une faute de l'utilisateur — répertoire inexistant, fichier de règles fautif, couleur inconnue — y ressort comme un message et non comme une trace d'exécution : ces messages sont écrits pour être lus, et douze lignes de pile les noyaient alors même que la ligne utile s'y trouvait, en tête.

Par programme , si l'on veut choisir l'apparence :

17.12 Demander un dossier

```
UMLGenerator.builder()
    .sourceDirectory("target/classes")
    .outputDirectory("diagrammes")
    .outputFormat(".svg")   // facultatif, .svg par defaut
    .build()
    .generate();
```

La forme d'origine, `outputFile("architecture.svg")`, reste disponible et n'écrit que le diagramme d'ensemble. C'est `outputDirectory` qui écrit toutes les vues.

18 Les boîtes que le code ne porte pas

18.1 Principe

Un diagramme n'est pas toujours le reflet exact du code. On veut y montrer une classe encore à écrire, ou une abstraction que le code réalise autrement — une hiérarchie de deux sous-classes explique parfois mieux qu'une classe unique à deux champs dont un seul est renseigné. Les diagrammes tenus à la main le font couramment, et s'en priver reviendrait à interdire de dessiner ce qu'on a en tête.

```
declare class StringRepresentation extends Representation
declare field StringRepresentation string : String
declare association Animal -> Chien 0..*
```

18.2 Ce que cela coûte, et comment on le paie

Une boîte déclarée ici échappe à tout ce qui fait la valeur de la bibliothèque : aucun compilateur ne la vérifie, aucun renommage ne la suit, et rien ne signalera le jour où le code aura pris un autre chemin. C'est exactement le défaut des diagrammes tenus à la main, et il serait malhonnête de le réintroduire en silence.

D'où la contrepartie, **non négociable** : une boîte déclarée **se voit**. Elle porte le stéréotype `«declared»` et un **contour pointillé**, si bien qu'un lecteur distingue au premier coup d'œil ce qui a du code derrière de ce qui n'en a pas. Un diagramme mixte qui ne marquerait pas la différence serait pire que les deux extrêmes.

18.3 Ce qui est refusé

Un nom introuvable — ni classe du diagramme, ni boîte déclarée — arrête la génération. Une association ne peut pas *partir* d'une classe réelle : son entité est construite à partir de son code, et une règle qui lui ajouterait un lien passerait par-dessus ce que ce code déclare, sans que rien ne le montre. Le lien se déclare alors depuis l'autre extrémité, ou par `@UMLAssociation` dans le code.

Un attribut déclaré n'a ni visibilité ni multiplicité : ce qui n'existe pas dans le code n'a pas d'encapsulation à montrer, et une visibilité inventée serait une affirmation de plus que rien ne garantit.

19 La vérification à la compilation

19.1 Principe

La bibliothèque embarque un **processeur d'annotations** (JSR 269), c'est-à-dire du code qui s'exécute à l'intérieur de `javac`. Il n'engendre aucun fichier : il lit les annotations et parle, en pointant la valeur en cause dans le fichier source.

19.2 Pourquoi c'est utile ?

Une annotation mal écrite est une **faute de code**, pas un incident d'exécution. La signaler à la compilation la met sous les yeux de son auteur, au moment où il l'écrit, plutôt

que quelques minutes plus tard au fond d'une pile d'exceptions — ou, pire, jamais : sur un diagramme silencieusement faux.

Le processeur ne réécrit aucune règle. Il construit les objets du modèle avec les valeurs lues dans l'annotation et traduit en erreur de compilation l'exception que le modèle lève déjà. Les deux niveaux de vérification ne peuvent donc pas diverger, et le message affiché par `javac` est mot pour mot celui que la génération aurait produit.

19.3 Deux gravités, et une règle pour les départager

— **Erreur** : *ce code ne décrit aucun diagramme.* La compilation s'arrête.

— **Avertissement** : *ce code décrit un diagramme, mais vraisemblablement pas celui que son auteur croit.* La compilation continue — l'annotation fait toujours foi, et la bibliothèque n'est pas juge des intentions. Mais taire l'écart laisserait l'utilisateur le découvrir sur l'image, sans le moindre indice sur sa cause.

19.4 Ce qui arrête la compilation

Situation	Raison
Cible d'un lien dépourvue de `@UMLClass`	La cible a été écrite exprès ; son absence du diagramme est un oubli.
Cible primitive (`int.class`) ou qui n'est pas une classe	Seule une classe du diagramme peut être une extrémité.
Multiplicité illisible (`"beaucoup"`) ou inversée (`"3..1"`)	Le vocabulaire des multiplicités est fermé.
Couleur inconnue de PlantUML	Vérifiée contre le catalogue réel du moteur de rendu, pas contre sa seule forme.
Texte fait uniquement d'espaces (nom, rôle, label, stéréotype, titre, `name` de `@UMLInclude`)	Produirait une étiquette invisible.
`@UMLInclude(name = ...)` et `targetRole` sur la même ligne	Deux noms pour la même ligne se contrediraient (section 2).
`@UMLPackage(name)` vide ou fait d'espaces	Un groupe est identifié par son nom ; le rectangle porterait une étiquette illisible.
`@UMLLayout` vers la classe elle-même, ou deux fois vers la même cible	Se placer par rapport à soi ne dit rien ; « en haut » puis « en bas » se contredisent.
`@UMLAssociationClass` dont une extrémité est la classe annotée	Le lien serait rattaché à la boîte même qui le qualifie.
`@UMLInclude` et `@UMLExclude` sur le même membre	Intention contradictoire (section 2).
`asAttribute = true` sur une méthode à paramètres, sur une méthode `void`, ou sur un attribut	La notation attribut retire les parenthèses : il ne reste que le nom et le type, et il faut que les deux existent — et un attribut en est déjà un.
`@UMLMultiplicity` et `@UMLAssociation` sur le même attribut	Deux multiplicités pour une seule propriété ; `targetCardinality` la donne déjà.
Annotation posée sur une classe dépourvue de `@UMLClass`	Le lien serait tracé depuis une boîte inexistante.
Dépendance d'une classe vers elle-même, ou deux fois la même dépendance	N'apprendrait rien, ou dessinerait deux traits superposés.

19.5 Ce qui produit un avertissement

Situation	Ce que l'auteur attendait, et n'aura pas
`@UMLInclude` sur un attribut que la `strategy` affiche déjà	La dérogation ne déroge à rien — sauf si elle **renomme** : renommer est un effet, et rien n'est alors signalé.
`@UMLExclude` sur un membre que la `strategy` masque déjà	Idem, dans l'autre sens.
`@UMLInclude` sur un attribut porteur de `@UMLAssociation`	Sous `ARROW`, l'attribut est dessiné comme un lien et rien ne l'y ramènera ; sous `ATTRIBUTE`, il est déjà affiché quelle que soit la stratégie. Dans les deux cas la dérogation d'affichage ne sert à rien — sous `ATTRIBUTE`, un `name` qui renomme la ligne reste en revanche un effet réel, et n'est pas signalé.
`@UMLInclude(name = ...)` reprenant le nom du code	Le membre est renommé vers le nom qu'il a déjà : le renommage n'a pas eu lieu, et l'auteur le chercherait sur l'image.
`@UMLExclude` sur un attribut porteur de `@UMLAssociation`	Ce qui disparaît est ce que l'association demandait : le **lien entre les deux boîtes** sous `ARROW`, la **ligne dans la boîte** sous `ATTRIBUTE`.
Paramètre de trait écrit sous `notation = ATTRIBUTE` (`type`, `label`, `qualifier`, `direction`, `sourceRole`, `sourceCardinality`)	Une ligne de boîte ne porte qu'un nom, un type et une multiplicité : ces valeurs ne seront dessinées nulle part. Le message les nomme une à une.
`stereotype` contenant déjà `« »`	Les guillemets seraient affichés en double.
Deux classes affichant la même étiquette	Deux rectangles que le lecteur ne peut pas distinguer ; `name` est là pour cela.
`@UMLDependency` vers une classe déjà atteinte par une `@UMLAssociation` **dessinée en trait**	Une association dit strictement plus qu'une dépendance ; le second trait n'ajoute que de l'encombrement. Sous `ATTRIBUTE`, aucun trait n'est tracé : rien n'est alors signalé.
Attribut de type collection sans `targetCardinality`	Ni le lien ni la ligne ne porteront de multiplicité. Seul le métier sait si c'est `0..*` ou `1..*`.
Multiplicité que le type contredit (une `List` pour `"0..1"`, un attribut simple pour `"1..*"`)	L'un des deux se trompe, et ce n'est pas au générateur d'en décider.
`target` absent du type de l'attribut	Souvent un copier-coller. Le lien sera tout de même dessiné vers la classe déclarée.
`@UMLDerived` sur un membre que le diagramme n'affiche pas	Une barre oblique ne se voit que si l'élément qu'elle marque est dessiné.
`@UMLMultiplicity` sur un membre que le diagramme n'affiche pas	Des crochets ne se voient que si le membre qu'ils suivent est dessiné.
`@UMLMultiplicity("")`	Une multiplicité vide est ce que porte un membre qui n'en a pas : l'annotation ne demande rien.
`@UMLDerived` sur un lien **dessiné en trait** et dépourvu de `targetRole`	La barre se pose devant le rôle cible ; sans rôle, rien ne peut la porter. Sous `ATTRIBUTE`, la ligne a toujours un nom : rien n'est signalé.
`@UMLNote` visant un membre que le diagramme n'affiche pas	La bulle n'a rien où s'accrocher et ne sera pas dessinée.
`@UMLNote` sur un attribut porteur	L'attribut est devenu un trait, et une note ne

Chacun de ces contrôles **se tait dès qu'il ne peut pas conclure** : une collection écrite sans paramètre de type (`List` nu) ne se voit jamais reprocher sa cible, puisque c'est exactement le manque que `target` vient combler. Un avertissement qui se trompe coûte plus cher que celui qui manque un cas : le premier apprend à ne plus les lire.

Deux exemptions, tirées de cas où le reproche était faux. Les **composants d'un record** échappent à tous les contrôles qui disent « ce membre n'est pas affiché » : un composant est dessiné quelle que soit la stratégie, et le déclarer invisible est un contresens. L'annotation qu'il porte est de surcroît recopiée par `javac` sur le champ *et* sur l'accesseur, si bien qu'un message désignait l'un des deux clones — donc une ligne que l'auteur n'a jamais écrite, et le disait deux fois.

Un `@UMLInclude` qui déroge à une règle `hide field` du projet n'est pas davantage « sans effet » : il en est le seul remède. Le contrôle ne regardait que la stratégie, et reprochait à la dérogation ce qui la justifiait.

19.6 Faire taire les avertissements

L'option `-Aumlg.warnings=none` les éteint tous. Les erreurs, elles, ne s'éteignent pas : elles disent qu'aucun diagramme ne peut être produit, ce dont personne ne peut vouloir être dispensé.

`none` est la seule valeur reconnue, et toute autre est **refusée**. Elle ne l'était pas : `NONE`, `off`, `false` laissaient les avertissements passer sans que rien ne signale la faute de frappe, et l'auteur croyait avoir fait taire le processeur en se demandant pourquoi il parlait encore.

Le silence porte par ailleurs sur ce que le processeur a trouvé dans le code, et non sur son propre état : un `-Aumlg.rules` désignant un fichier introuvable reste signalé même sous `none`. L'option demande « ne me dis pas ce que tu as trouvé chez moi », ce qui ne peut pas vouloir dire « ne me dis pas que tu es mal configuré » — un fichier de règles hors de portée désarme six contrôles, et le taire ferait passer un processeur aveugle pour un processeur satisfait.

19.7 Donner le fichier de règles au compilateur

Six avertissements dépendent de la stratégie appliquée à la classe : ceux qui disent « ce membre n'est pas affiché » ou « il l'est déjà »[1]. Quand la classe déclare sa stratégie, le processeur conclut seul. Quand elle s'en remet au projet — le défaut `PROJECT` —, la réponse est écrite dans `umlg.rules`, un fichier que `javac` ne sait pas trouver : il vit à la racine des classes compilées, et rien, dans une compilation, ne dit ce que la construction fera de leur résultat.

Le processeur **se tait** alors sur ces six contrôles, plutôt que de parler au hasard. C'est la seule chose que la bibliothèque taise, et une option suffit à la lui rendre :

```
<compilerArgs>
    <arg>-Aumlg.rules=src/main/resources/umlg.rules</arg>
</compilerArgs>
```

Le processeur lit alors les mêmes règles que la génération, et retrouve la parole : aucun des six n'est perdu. Trois situations, trois gravités :

1. `@UMLInclude` et `@UMLExclude` sans effet, et `@UMLDerived`, `@UMLMultiplicity`, `@UMLNote` ou `@UMLConstraint` posés sur un membre que le diagramme n'affiche pas.

— **option absente** : le cas ordinaire. Le processeur se tait sur ce qu'il ne peut pas savoir, et rien d'autre ne change ;
— **fichier introuvable** : **avertissement**. Le code reste juste, mais son auteur attend des vérifications qu'il n'aura pas ;
— **fichier fautif** : **erreur**. Le même fichier arrêterait la génération pour la même raison ; le dire ici le dit plus tôt, ce qui est tout l'intérêt du processeur.

Cette option ne change jamais le diagramme produit, et n'ajoute ni ne retire aucune *erreur* : toutes celles que le processeur connaît sont indépendantes de la stratégie, et signalées dans tous les cas.

20 Choix de conception : ce qui est hors périmètre

Certaines possibilités de PlantUML ont été délibérément écartées. Les mentionner explicitement évite de les réintroduire par inadvertance et documente l'arbitrage retenu.

20.1 Fonctionnalités écartées

— **Style fin des liens** : couleur, épaisseur, pointillés, longueur des flèches. Gain purement esthétique pour un coût élevé en surface d'API.
— **Navigabilité explicite** (`--` contre `-->` contre `<-->`) : la flèche déduite automatiquement convient dans la quasi-totalité des cas.
— **Pastille de stéréotype** (`«(S,#FF7700) Service»`) et stéréotypes multiples : le paramètre `stereotype` reste une chaîne simple.
— **Imbrication de paquetages** et variantes de conteneurs (`namespace`, `folder`, `frame`, `together`) : un seul niveau de regroupement suffit à diviser la complexité visuelle.
— **Notes rattachées à un lien** et notes flottantes non ancrées.
— **Directives brutes** `skinparam` et thèmes PlantUML : exposer ces réglages reviendrait à réécrire PlantUML en syntaxe Java.

20.2 Ce qui relève de l'analyse automatique, non des annotations

Les éléments suivants sont déduits du code compilé et ne nécessitent aucune annotation : la nature de l'entité (classe, classe abstraite, interface, énumération, type annotation, *record*), les liens d'héritage lus dans `extends` et `implements`, les littéraux d'une énumération, le modificateur `static`, la visibilité de chaque membre, et les signatures de méthodes (type de retour, types des paramètres, et leurs noms lorsque le projet est compilé avec `-parameters`).

Deux s'y ajoutent, qui touchent à la notation UML elle-même. Les **paramètres de type** d'une classe générique sont écrits à la suite de son nom — `Boite<T>`, `Table<K, V extends Number>` — avec leurs bornes, celles-ci passant par les règles de nommage du projet comme n'importe quel autre type. UML les dessine dans un cartouche au coin de la boîte, que PlantUML ne sait pas produire ; les écrire à la suite dit la même chose. Ils restent distincts du nom, qui désigne la boîte : c'est par lui qu'une vue la nomme, et les y coller rendrait `view "X" = Boite` introuvable dès que `Boite` devient générique.

Une **opération sans corps** — `abstract` au sens de Java, ou déclarée par une interface sans être `default` — est écrite en italique, comme UML l'exige. C'est la seule chose qui distingue à l'œil un contrat d'une implémentation, et le modificateur suffit à le savoir :

une méthode d'interface est abstraite sans porter le mot-clé, une `default` ne l'est pas bien qu'elle vive au même endroit.

20.3 Ce qui ne peut pas être déduit, et pourquoi

La lecture ayant lieu après compilation (section 1.1), deux informations ont déjà disparu et ne sont donc *pas* déductibles : les **valeurs d'initialisation des attributs**, et le **corps des méthodes** — d'où `@UMLDependency`, seule façon de déclarer un lien qui ne se lit que dans un appel.

Une troisième, longtemps rangée ici, n'y a pas sa place : le **contenu des collections**. Les paramètres de type survivent dans les signatures déclarées (section 1.1), et la bibliothèque les lit. C'est pourquoi `target` n'est plus obligatoire sur `@UMLAssociation` : un `List<Commande>` nomme déjà sa cible, et la redéclarer ne ferait qu'ouvrir un écart possible entre l'annotation et le code.

Ce qui reste réellement indéductible, et que `@UMLAssociation` vient combler, est la **multiplicité** : rien dans `Set<String>` ne dit `0..*` plutôt que `1..*`. Cela relève du métier, et le deviner produirait un diagramme faux une fois sur deux.

Un quatrième cas se règle autrement : le **nom des paramètres de méthode** n'a pas disparu à cause d'un choix de la bibliothèque, mais parce que `javac` ne l'écrit dans les `.class` que sur demande. Il ne s'agit donc pas d'une déduction impossible mais d'une option de construction du projet analysé, `-parameters`, décrite en section 2.2. Sans elle, la signature se limite aux types ; aucune annotation ne vient combler ce manque, car le compilateur sait déjà le faire.